



GENERACIÓN DE DATOS SINTÉTICOS DE COLONIAS DE BACTERIAS PARA LA POSTERIOR IMPLEMENTACIÓN DE ALGORITMOS YOLO

DANIELA MORENO RENCORET

**Tesis para optar al título de Ingeniero Civil en Informática y
Telecomunicaciones**

Profesor guía: Martín Gutiérrez Pescarmona

**FACULTAD DE INGENIERÍA Y CIENCIAS ESCUELA DE
INFORMÁTICA Y TELECOMUNICACIONES**

**Santiago, Chile
2022**

GENERACIÓN DE DATOS SINTÉTICOS DE COLONIAS DE BACTERIAS PARA LA POSTERIOR IMPLEMENTACIÓN DE ALGORITMOS YOLO

DANIELA MORENO

**Tesis para optar al título de Ingeniero Civil en Informática y
Telecomunicaciones**

**Profesor guía: Martín Gutiérrez Pescarmona
Comité: Nicolás Hidalgo Castillo
Comité: Víctor Reyes Rodríguez**

**FACULTAD DE INGENIERÍA Y CIENCIAS
ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES**

**Santiago, Chile
2022**

📧 Daniela Moreno
✉ daniela.moreno1@mail.udp.cl

Dedicado a mi familia y amistades que me han brindado su apoyo.

Agradecimientos

Agradezco a todas las personas que me han apoyado en este proceso. En particular a mi familia y amistades por su confianza y soporte emocional. Como también a los académicos por entregarme su conocimiento y tiempo a lo largo de la carrera. Sobre todo, al profesor Martín por darme la oportunidad de participar en este proyecto.

Resumen

El campo de la microbiología representa un desafío a la hora de diseñar sistemas que requieren de una gran cantidad de datos. Ello se debe principalmente a que el acceso a los datos es condicionado a tener las herramientas necesarias, tales como microscopios, cámaras, cultivos bacterianos, entre otros. Además, requiere tener los conocimientos básicos de bacteriología. En el presente trabajo se plantea una solución para la obtención de un conjunto de imágenes sintéticas, en donde se empleará un motor de videojuegos para generar imágenes mediante una simulación que represente una colonia de bacterias. A su vez, se tendrán las anotaciones de forma automática de la posición de cada bacteria de la simulación enmarcadas por un rectángulo 2D. Finalmente, se revisará la calidad y compatibilidad de los datos con la técnica de detección de objetos conocida como YOLO.

Abstract

The field of microbiology represents a challenge when designing systems that require a large amount of data. This is mainly due to the fact that access to data is conditioned on having the necessary tools, such as microscopes, cameras, bacterial cultures, among others. In addition, it requires basic knowledge of bacteriology. In the present work, a solution is proposed to obtain a set of synthetic images, where a video game engine will be used to generate images through a simulation that represents a colony of bacteria. In turn, there will be annotations automatically of the position of each bacterium in the simulation framed by a 2D rectangle. Finally, the quality and compatibility of the data with the object detection technique known as YOLO will be reviewed.

Contenido

Resumen	1
Abstract	3
Lista de tablas	9
Lista de figuras	12
Capítulo 1. Introducción	1
1.1. Contextualización y motivación	1
1.2. Objetivos	3
1.2.1. Objetivo general	3
1.2.2. Objetivos específicos	3
1.3. Solución preliminar	4
1.4. Resultados esperados	5
Capítulo 2. Estado del arte	7
2.1. Identificación y caracterización bacteriana	7
2.1.1. Tamaño	7
2.1.2. Forma y agrupación	8
2.1.3. Tinciones	10
2.2. Redes neuronales artificiales	10
2.2.1. Algoritmos de aprendizaje automatizado	12
2.2.2. Redes neuronales convolucionales	13
2.3. YOLO	14
2.3.1. Arquitectura	15
2.3.2. Variantes de YOLO	15
2.4. Generación de datos artificiales	16
2.4.1. Brecha de dominio	16
Capítulo 3. Metodología de trabajo	19
3.1. Investigación e hipótesis	19
3.2. Diseño	19
3.3. Experimentación y análisis de resultados	20

Capítulo 4. Solución propuesta	21
4.1. Diseño	21
4.1.1. Diseño de la solución	22
4.1.2. Definición de bacterias y tipos de tinciones	22
4.1.3. Definición de métricas para los experimentos	22
4.2. Implementación	24
4.2.1. Recopilación de datos reales	24
4.2.2. Creación de materiales	24
4.2.3. Implementación del generador	27
4.2.4. Conversión de formato	29
4.2.5. Implementación de YOLOv4	30
4.2.6. Implementación de YOLOv5	32
4.3. Experimentación	33
4.3.1. Experimentos de sensibilidad de parámetros de entre- namiento	33
4.3.2. Comparación de versiones YOLO	35
4.3.3. Pruebas de aleatorización de dominio	35
4.3.4. Validación de los <i>datasets</i>	40
Capítulo 5. Análisis de resultados	43
5.1. Análisis de sensibilidad de parámetros de entrenamiento . . .	43
5.2. Análisis de comparación de versiones YOLO	44
5.3. Análisis de técnicas de aleatorización de dominio	45
5.4. Análisis de calidad de los <i>datasets</i>	45
5.4.1. Curvas de precisión versus confianza	46
5.4.2. Curvas de precisión versus <i>recall</i>	46
5.4.3. Curvas de <i>recall</i> versus confianza	47
5.4.4. Curvas de F1 versus confianza	48
5.4.5. Matrices de confusión	49
5.4.6. Observaciones generales	50
Capítulo 6. Conclusión y trabajos futuros	53
Referencias bibliográficas	55
Anexo A. Primer anexo	61
A.1. Conversión de archivos de anotaciones para YOLOv4	61
A.2. Creación de archivos de entrenamiento para YOLOv4	62

A.3. Conversión de anotaciones y estructuración de carpetas para YOLOv5	63
A.4. Aleatorizadores personalizados	65
A.4.1. Aleatorizador de posición de objetos principales orientada a la agrupación	65
A.4.2. Aleatorizador de color e intensidad de la iluminación	68
Anexo B. Segundo anexo	71
B.1. Muestra de imágenes del <i>dataset</i> sintético.	72
B.2. Gráficas de comparación entre <i>dataset</i> real, sintético y mixto.	73
B.2.1. Gráficas de precisión versus confianza	73
B.2.2. Gráficas de precisión versus <i>recall</i>	75
B.2.3. Gráficas de <i>recall</i> versus confianza	76
B.2.4. Gráficas de F1 versus confianza	78
B.2.5. Matrices de confusión	80

Lista de tablas

4.1. Comparación entre cantidad de épocas.	34
4.2. Comparación entre tamaños de <i>batch</i>	34
4.3. Comparación entre tamaños de imágenes.	35
4.4. Comparación entre YOLOv4 y YOLOv5.	35
4.5. Tabla de resultados del <i>dataset</i> real.	41
4.6. Tabla de resultados del <i>dataset</i> sintético.	42
4.7. Tabla de resultados del <i>dataset</i> mixto.	42

Lista de figuras

2.1. Morfología: 1. cocos; 2. diplococo; 3. estreptococos; 4. cocos en racimos; 5. cocos en tetradas; 6. cocobacilos; 7. bacilos; 8. bacilos bordes redondeados; 9. bacilos bordes rectos; 10. bacilos fusiformes; 11, 12. bacilos curvos; 13 al 15. espiroquetas [1].	9
2.2. Esquema de la estructura de una neurona artificial [2].	10
2.3. Estructura del modelo MLP.	12
2.4. Capa totalmente conectada en una CNN [3].	14
2.5. Arquitectura de YOLO [4].	15
2.6. Historia de YOLO [5].	16
4.1. Flujo de la solución propuesta.	22
4.2. Diseño de objetos: bacterias con forma de espiral, cilíndrica y esférica.	25
4.3. Modelo de bacilo en tres poses distintas de una animación.	26
4.4. Muestra del material diseñado para las bacterias.	26
4.5. Objetos de fondo y textura de ejemplo.	27
4.6. Componentes en la escena del simulador.	28
4.7. Estructura de carpetas para YOLOv5.	32
4.8. Gráfica de distancia y porcentaje de agrupación versus mAP50 para todas las formas.	38
4.9. Muestra de los fondos de la experimentación para la forma de los bacilos: 1. fondo solo con variación de color; 2. fondo con variación de color y HUE; 3. fondo con variación de texturas; 4. fondo con variación de texturas y HUE.	39
4.10. Gráfica de comparación de los tipos de fondos versus mAP50 para todas las formas.	39
4.11. Gráfica de comparación de los modificadores versus mAP50 para todas las formas.	40
5.1. Gráfica de variación de dominio que entrega el mAP50 general para todas las formas del modelo. Este señala que el mejor <i>dataset</i> es el mixto, seguido del real y, finalmente, el sintético, además se observa que el desempeño de los dominios sintético y mixto detectando bacterias sintéticas es bastante alto.	50

B.1. Selección de escenas aleatorias utilizadas en la fase de entrenamiento para la comparación de <i>datasets</i>	72
B.2. Curva de precisión versus confianza para el <i>dataset</i> real. . . .	73
B.3. Curva de precisión versus confianza para el <i>dataset</i> sintético. . .	74
B.4. Curva de precisión versus confianza para el <i>dataset</i> mixto. . . .	74
B.5. Curva de precisión versus <i>recall</i> para el <i>dataset</i> real.	75
B.6. Curva de precisión versus <i>recall</i> para el <i>dataset</i> sintético. . . .	75
B.7. Curva de precisión versus <i>recall</i> para el <i>dataset</i> mixto.	76
B.8. Curva de <i>recall</i> versus confianza para el <i>dataset</i> real.	76
B.9. Curva de <i>recall</i> versus confianza para el <i>dataset</i> sintético. . . .	77
B.10. Curva de <i>recall</i> versus confianza para el <i>dataset</i> mixto.	77
B.11. Curva de F1 versus confianza para el <i>dataset</i> real.	78
B.12. Curva de F1 versus confianza para el <i>dataset</i> sintético.	78
B.13. Curva de F1 versus confianza para el <i>dataset</i> mixto.	79
B.14. Matriz de confusión para el <i>dataset</i> real.	80
B.15. Matriz de confusión para el <i>dataset</i> sintético.	81
B.16. Matriz de confusión para el <i>dataset</i> mixto.	82

Capítulo 1

Introducción

En este capítulo se indicará la información relacionada con el entendimiento del proyecto, tales como contexto, motivación, objetivos, solución preliminar y resultados esperados.

1.1. Contextualización y motivación

El área de la inteligencia artificial (IA) se ha transformado debido al aumento de la potencia computacional y a los conjuntos de datos de gran escala proporcionados de forma pública. Con esto surgen tecnologías de aprendizaje por computadora (ML) como la clasificación, predicción y generación de datos. Como también, el desarrollo de métodos de aprendizaje profundo (DL) que utilizan redes neuronales para resolver problemas complicados con mayor precisión.

La utilización de métodos de DL permite tareas claves en la visión por computadora, como detección y reconocimiento de objetos, reconocimiento facial, reconocimiento de acción/actividad y estimación de pose humana [6]. Para conseguir este tipo de aplicaciones, uno de los factores claves es poseer un conjunto de datos etiquetado de gran volumen y calidad.

Algoritmos de tipo You Only Look Once (YOLO) utilizan el aprendizaje profundo para ofrecer detección de objetivos a tiempo real, permitiendo la detección de múltiples objetos con una alta precisión [7, 8]. El resultado de esto, puede combinarse con redes neuronales convoluciones (CNN) co-

mo Simple Online and Realtime Tracking with a Deep Association Metric (DeepSORT) [9] para lograr un flujo de datos que entregue detección y seguimiento de objetos. Esto permite aplicaciones de interés para el campo la microbiología, ya que con la identificación de una colonia y su cambio en el tiempo se puede contar la cantidad de bacterias para realizar una caracterización de las dinámicas físicas y tener conocimiento de sus funcionalidades bacterianas, tales como: crecimiento bacteriano, tiempos de aparición y tipo de división.

En la aplicación de este tipo de técnicas se requiere de una gran cantidad de imágenes y anotaciones para realizar un entrenamiento adecuado de aprendizaje profundo. Esto puede tomar mucho tiempo y el etiquetado manual es propenso a errores. Por ello, es de relevancia la implementación de un programa que permita la generación de un *dataset*¹ de colonias de bacterias sin la necesidad de tener acceso a un laboratorio o conocer metodologías para la toma de muestra de microorganismos.

Dentro del área de la Inteligencia Artificial y el aprendizaje automático, la generación de datos sintéticos es un campo reciente. Este puede dividirse en dos tipos, el primero es la generación de datos a partir de un conjunto de datos reales. En este se utilizan modelos estadísticos o aplicaciones de inteligencia artificial como las Redes Generativas Antagónicas (GANs). Por otro lado, están las técnicas de generación sin utilizar datos reales, los cuales son datos sintéticos creados por modelos existentes o conocimiento previo del analista [10].

Los conjuntos de datos sintéticos que son generados programáticamente pueden ayudar inmensamente en el campo de aprendizaje automático. Estos conjuntos de datos no son recopilados por ninguna encuesta o experimento de la vida real. Su finalidad principal, por tanto, es la de ser lo suficientemente flexibles y ricos para ayudar en la realización de experimentos con varias clasificaciones, segmentaciones y algoritmos de detección de objetos [11]. Además, si el analista tiene un conocimiento de alta calidad del dominio, se pueden obtener resultados con los datos sintéticos que son similares a los que se tendrían con datos reales.

Las técnicas de generación de datos sintéticos a partir de una simulación reducen drásticamente el esfuerzo de anotación y logran una alta precisión. Dado que al generar una imagen se tiene la posición y el tipo de objeto con claridad. Con esto, la motivación de este trabajo es explorar el uso de datos

¹Dataset: conjunto de datos.

sintéticos en el área de la microbiología e impulsar las técnicas de aprendizaje profundo para el contexto de las colonias de bacterias.

Adicionalmente, se destaca el paquete de Unity² llamado Perception, módulo especializado en generar gran cantidad de datos con anotaciones para el entrenamiento de aprendizaje automático [12] que incluye herramientas de base para la aleatoriedad, toma de imágenes y anotaciones del *dataset* objetivo. Para utilizar este paquete se necesitan incluir los modelos, materiales y texturas adecuados al tipo de imágenes que se desee generar. En este caso, se precisa de una investigación acerca de las características fenotípicas de las bacterias objetivas y su entorno.

La accesibilidad a las imágenes de colonias de bacterias se ve limitada a contar con los implementos requeridos para la captura de imágenes, tales como la necesidad de un microscopio. Pero, además, existe un problema en el área de la biología sintética, en donde trabajan personas con una perspectiva más abstracta orientada a la programación y las simulaciones. Las cuales, comúnmente, no cuentan con los medios para la toma de muestras o la accesibilidad a datos de calidad que sean públicos. Por ello, la creación de un programa que genere de forma automática imágenes de este tipo impulsaría a las investigaciones y los pipelines³ de desarrollo orientado al campo de la bioinformática y simulación de microbiología/biología sintética.

1.2. Objetivos

1.2.1. Objetivo general

Generar un conjunto de datos artificiales con anotaciones para la posterior implementación de algoritmos de tipo YOLO sobre colonias de bacterias.

1.2.2. Objetivos específicos

- Estudiar los factores fenotípicos y patrones de comportamiento que describen a una colonia de bacterias, tales como: forma, color, textura, tipo de agrupación y división celular.
- Implementar un generador de imágenes y anotaciones que incluya factores de aleatoriedad y diseño orientado a los procesos y comportamiento propio de una colonia de bacterias.

²Unity: motor de videojuego multiplataforma.

³Pipeline: flujo de trabajo.

- Implementar distintas versiones del algoritmo YOLO y realizar el preprocesamiento necesario para utilizar el *dataset* sintético en el entrenamiento de la red neuronal.
- Validar la solución y la calidad de las imágenes generadas a partir de la detección de bacterias en imágenes reales.

1.3. Solución preliminar

Se propone utilizar el motor de videojuegos Unity junto con el paquete Perception [12] dedicado a la generación sintética de datos. Este paquete contiene métodos para variar la posición, orientación, fondo, color y textura de los objetos en un escenario tridimensional (3D). Cada *frame*⁴ de simulación significará una captura de imagen junto con las anotaciones de la ubicación, tamaño y clase de todos los objetos en el escenario. A su vez, los objetos variarán sus parámetros a base de programas que definen su comportamiento y transformación a lo largo de la simulación. Por ende, la aleatoriedad del *dataset* generado se debe adecuar al objeto que se desee representar.

Antes de diseñar el material que se utilizará, es necesario realizar un estudio sobre el área de biología/microbiología. El cual, será fundamental para codificar la aleatoriedad en las características de los objetos orientada al dominio de la biología. En primer lugar, la variación de posición y rotación de las bacterias se tomará en cuenta su forma, tipo de agrupación y división celular empleada. Por otro lado, en la variación de colores y texturas se utilizarán gamas similares a las que se obtienen en las tinciones utilizadas en la bacteriología. Luego de que los resultados obtenidos sean congruentes con el estudio realizado acerca del comportamiento de bacterias y su caracterización fenotípica, se procederá a recopilar los *datasets* y efectuar el preprocesamiento necesario.

Para la utilización de YOLO, al igual que en la mayoría de tecnologías de detección de objetos, se requiere tener un formato de anotación y estructuración de datos específico. Debido a esto, se creará un programa que automatice la limpieza, estructuración y conversión de formato. Otro punto a considerar es que el entrenamiento de este tipo de algoritmos necesita un poder computacional considerable, principalmente relacionadas con la tar-

⁴Frame: cuadro en un vídeo o imagen en movimiento.

jeta de vídeo (GPU). Por consiguiente, se utilizará Jupyter Notebook⁵, el cual puede ser ejecutado localmente o en línea mediante el uso de servicios de nube como Google Colab⁶, AWS SageMaker⁷, Paperspace Gradient⁸ o Azure ML⁹.

Posterior a la implementación, se busca que el entrenamiento mediante datos generados de forma sintética tenga la calidad suficiente para que YOLO logre detectar e identificar las bacterias en imágenes reales de bacterias. Dado esto, se evaluará la calidad de la solución a partir del porcentaje de acierto en la detección de formas bacterianas por la red neuronal en el conjunto de prueba. Este conjunto estará constituido con imágenes reales etiquetadas manualmente.

Adicionalmente, una de las métricas a considerar a la hora de la validación es la cantidad mínima de bacterias reales que debe detectar la red neuronal, ya que esta debe ser suficiente para decir que es una cantidad representativa de la colonia.

1.4. Resultados esperados

Se espera que la implementación de Unity sea capaz de generar datos para el área de microbiología con una calidad lo suficientemente alta para que las redes neuronales entrenadas por el *dataset* sintético logren identificar bacterias en imágenes reales. La implementación se entregará por medio de un enlace a un repositorio público, donde estará el proyecto de Unity y los Jupyter Notebook para el preprocesamiento, entrenamiento y utilización del *dataset* sintético. Adicionalmente, se espera que el proyecto y los nuevos conjuntos de datos aporten en el avance de la identificación y caracterización de colonias de bacterias con métodos de aprendizaje profundo.

⁵Jupyter Notebook: plataforma computacional interactiva que permite combinar código en vivo, ecuaciones, texto narrativo, visualizaciones, gráficos, etc.

⁶<https://colab.research.google.com>

⁷<https://aws.amazon.com/es/sagemaker>

⁸<https://www.paperspace.com/gradient>

⁹<https://studio.azureml.net>

Capítulo 2

Estado del arte

En esta sección se revisa el marco teórico necesario para entender el proyecto a realizar, como lo son: la identificación y caracterización bacteriana, las redes neuronales artificiales, YOLO y la generación de datos. Además, se incluyen los trabajos relacionados con la generación de datos y cómo cada uno enfrenta el problema de la brecha entre los datos reales y los datos artificiales.

2.1. Identificación y caracterización bacteriana

Para identificar y caracterizar a las bacterias es necesario estudiar acerca de sus propiedades estructurales, tales como: tamaño, forma y tipo de agrupación. Estas constituyen la morfología propia de las células bacterianas [13]. A su vez, también es importante saber sobre las técnicas de tinción que se utilizan para visualizar microorganismos con mayor facilidad en el microscopio.

2.1.1. Tamaño

El tamaño es una característica importante a la hora de identificar a las bacterias, este suele ser menor de $10\ \mu m$ y puede variar dependiendo de la forma de la especie. Un ejemplo es la E. Coli, bacteria con forma de bacilo que mide $0.5\ \mu m$ de ancho y $3\ \mu m$ de largo. Por lo mencionado anteriormente, es necesario de un microscopio para la visualización de las bacterias [14].

2.1.2. Forma y agrupación

Las bacterias pueden tener una gran variedad de formas, aunque a menudo se encuentran en cocos (esféricas y ovaladas) o bacilos (cilíndrica o de bastones; rectos o curvos). También existen un número significativo de bacterias con formas de bastones largos (espirales o hélices).

Un factor importante a considerar es que las bacterias pueden mantenerse unidas unas con otras después de la división celular, pero conservando siempre la independencia celular. Esto produce el fenómeno conocido como agrupación [1].

A continuación, se definen las principales formas, características y agrupaciones que pueden tomar las bacterias:

- **Oval o esfera:** se llaman cocos y pueden existir como células individuales o estar asociadas en agrupaciones. Por un lado, cuando la bacteria se divide manteniéndose unidas en pares, se les conoce como diplococos. Si esta división se repite en un mismo plano se forman cadenas llamadas estreptococos. Por otra parte, si se divide en dos planos perpendiculares, forman grupos de cuatro células llamadas tetracocos. Finalmente, si se divide en tres planos perpendiculares, forman agrupaciones cuboidales del orden de ocho cocos o más, se llaman sarcinas [13,14].
- **Cilindro o bastón:** se denominan bacilos y pueden tener distintas variaciones en su forma. Por ejemplo, unas pocas bacterias en forma de bastón, los vibriones, se curvan para formar comas distintivas o espirales incompletas. Del mismo modo, hay bacilos muy cortos que se les conoce como coco bacilos [13,15]. Este tipo de forma tienden a agruparse con menor frecuencia que en el caso de las formas cocoides. Las agrupaciones que pueden tomar son las siguientes: en primer lugar, si dos bacilos están unidos por la división en un mismo plano se llaman diplobacilos, mientras que si se divide múltiples veces en un plano en modo de cadenas se llaman estreptobacilos.
- **Espiral o helicoidal:** las células con forma de espiral reciben el nombre de espirilos si son rígidos y espiroquetas si son flexibles. A su vez, suelen tener gran variedad de longitud, número de espiras y amplitud [13,15]. En cuanto a la agrupación, son células individuales y más o menos aisladas.

- **Otras formas:** las morfologías forman un amplio rango, en donde algunas formas son más comunes que otras. Entre las formas menos comunes se encuentran las bacterias con forma de estrella, cuadradas o filamentosas [16].

Seguidamente, en la figura 2.1 se pueden apreciar gráficamente algunas de las formas y agrupaciones descritas anteriormente.

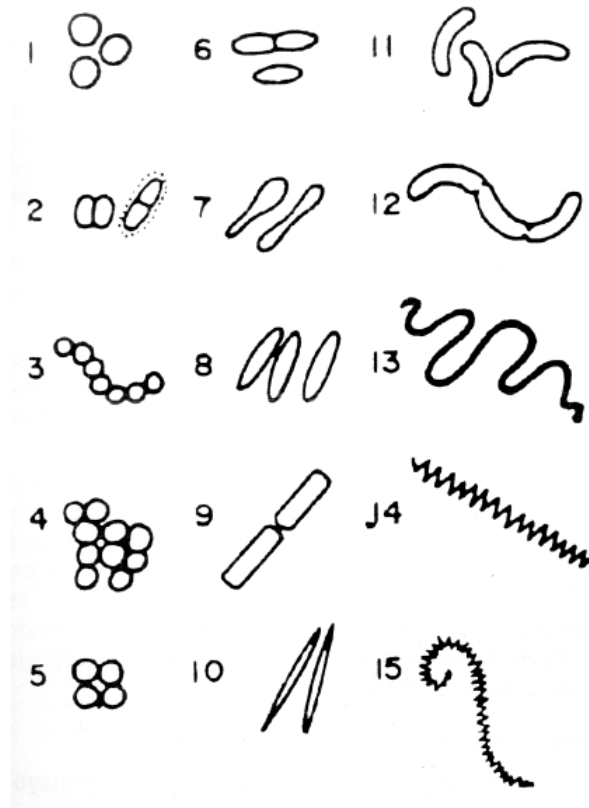


Figura 2.1. Morfología: 1. cocos; 2. diplococo; 3. estreptococos; 4. cocos en racimos; 5. cocos en tetradas; 6. cocobacilos; 7. bacilos; 8. bacilos bordes redondeados; 9. bacilos bordes rectos; 10. bacilos fusiformes; 11, 12. bacilos curvos; 13 al 15. espiroquetas [1].

2.1.2.1. Características especiales

Algunas especies presentan otras características adicionales, como es el caso de los flagelos, cápsulas y esporas. Ellas también forman parte de la morfología de las bacterias [13].

2.1.3. Tinciones

Las tinciones son procedimientos para visualizar diferentes microorganismos que son transparentes en el microscopio. Por lo que las tinciones representan un punto importante al identificar y caracterizar visualmente a las bacterias.

Dentro de los principales métodos de tinción se encuentra la tinción GRAM. Esta es una coloración diferencial que puede clasificar según el resultado en bacterias gram positivas o gram negativas. Las primeras se tiñen de un color azul violeta y las segundas adquieren un color rosado o rojo según el tipo de envoltura celular que presente la bacteria [1].

2.2. Redes neuronales artificiales

Las redes neuronales son la unidad funcional del aprendizaje profundo, estas imitan el comportamiento del cerebro humano en la forma en que almacena información, aprende y reconoce patrones. Gracias a esta tecnología se pueden utilizar a las redes neuronales para resolver problemas complejos.

La arquitectura de una red neuronal se compone de neuronas artificiales o perceptrones. Estas son unidades individuales que procesan un conjunto de entradas y generan una respuesta o salida única.

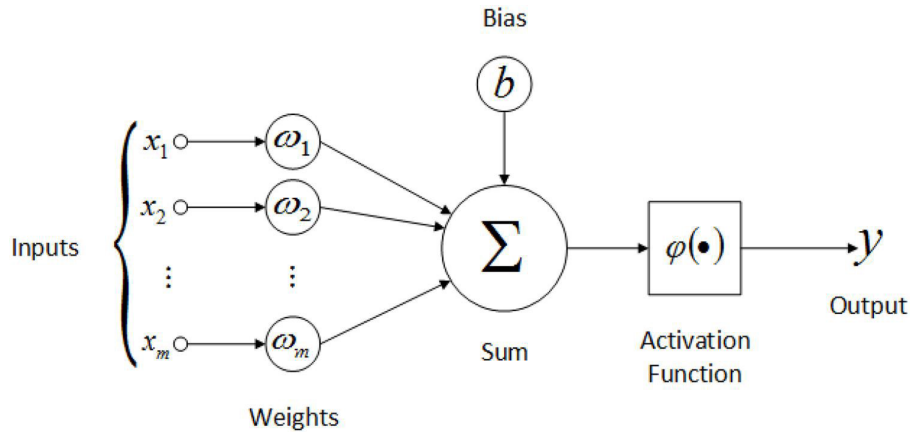


Figura 2.2. Esquema de la estructura de una neurona artificial [2].

Con base en la figura 2.2 se definen los componentes de las neuronas artificiales:

- **Señales de entrada:** las señales de entrada corresponde al conjunto de características $(x_1, x_2, \dots, x_m)$ que ingresa al sistema para realizar

el aprendizaje. Esta puede ser, por ejemplo, los vectores de píxeles de una imagen.

- **Conjunto de pesos:** a las señales de entrada se les introduce la multiplicación escalar con la matriz de pesos $(w_1, w_2, \dots, w_m)$ con el objetivo de dar importancia a las características más contribuyentes en el aprendizaje.
- **Bias:** corresponde al parámetro (b_k) que ingresa a la función suma para cambiar el valor producido por la función de activación. Su operación es similar al de una constante en una función lineal
- **Función suma:** se denota con el símbolo $(\sum)$ y su función es combinar múltiples entradas en un valor de salida para que se pueda utilizar la función de activación. Se realiza mediante la suma de todas las entradas a la función de suma o transferencia.
- **Función de activación:** el resultado obtenido de la función suma se ingresa en la función de activación $(\phi(.))$, la cual ayuda a introducir la no linealidad en el funcionamiento de los perceptrones. Sin esto, la salida solo sería una combinación lineal de valores de entrada.
- **Señal de salida:** es el valor resultante (y_k) generado por la neurona.

Cuando varias de estas neuronas se apilan juntas en filas, constituyen una capa, y las capas múltiples apiladas una al lado de la otra se conocen como redes neuronales de avance, o perceptrones multicapa (MLP). El cual es un modelo de aprendizaje profundo por excelencia denominado *feedforward*, ya que la información fluye a través de la red y ajusta sus pesos hacia atrás en la dirección que daría un resultado correcto [17].

En la figura 2.3 se ve la estructura de una estructura MLP con 3 entradas, 2 salidas y una capa oculta de 4 neuronas.

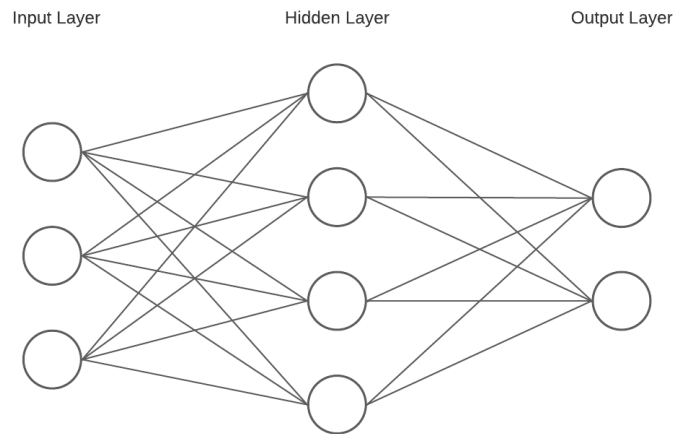


Figura 2.3. Estructura del modelo MLP.

A continuación, se describen los componentes de la estructura MLP:

- **Capa de entrada:** es la única capa visible que transmite la información completa sin ningún cálculo. Esta suele provenir desde señales de entrada (sensores) o de algún sistema de procesamiento de bajo nivel. Es decir, estos son los datos que se introducen en el modelo.
- **Capa oculta:** las capas intermedias se encargan de hacer los cálculos y extraer las características de los datos. Además, puede haber varias capas ocultas interconectadas que busquen características escondidas.
- **Capa de salida:** la capa de salida toma la información de las capas ocultas y llega a una clasificación, detección o interpretación basada en los aprendizajes del modelo.

2.2.1. Algoritmos de aprendizaje automatizado

Las redes neuronales artificiales utilizan un algoritmo adaptativo, también conocido como algoritmo de aprendizaje automatizado, para modificar sus parámetros hacia un comportamiento deseado [17].

Existen varios tipos de aprendizaje automático, la mayoría de estos se pueden dividir en dos categorías:

- **Aprendizaje supervisado:** requiere de tener ejemplos de una salida correcta para ciertas entradas mediante el uso de un conjunto de datos

con etiquetas de la ubicación para cada uno de los objetos que se desee reconocer [18].

- **Aprendizaje no supervisado:** en este tipo de aprendizaje se experimenta con un conjunto de datos que contiene muchas características y luego se aprenden propiedades útiles de la estructura. Por lo general se quiere aprender toda la distribución de probabilidad que generó un conjunto de datos, ya sea explícitamente como en la estimación de densidad o implícitamente para tareas como síntesis o eliminación de ruido. Otro caso de aplicación es el *clustering*. Este consiste en dividir el conjunto de datos en ejemplos similares [17].

2.2.2. Redes neuronales convolucionales

Las redes neuronales convolucionales (CNN) son un tipo de red neuronal que emplea una operación matemática llamada convolución. Es decir, las redes convolucionales son básicamente redes neuronales que utilizan la convolución en lugar de la multiplicación general de matrices en al menos una de sus capas [17]. Gracias a esto, se logran realizar tareas como análisis de imágenes, procesamiento de lenguaje natural y problemas complejos de clasificación de imágenes.

Una capa típica de una CNN consta de tres etapas principales. En la primera etapa, la capa realiza convoluciones en paralelo para producir un conjunto de activaciones lineales. En estos procesos de convolución, el tamaño de la imagen se reduce mediante el detector de características (filtro) que convierte la imagen en un mapa de características convolucionada.

Luego, en la segunda capa, cada activación lineal pasa por una función de activación no lineal, la cual se encarga de definir la salida de una neurona artificial en términos de la entrada dada. Esta salida pasa a la entrada de la siguiente capa neuronal [19]. Comúnmente se suele utilizar la función de activación lineal rectificada (ReLU). El funcionamiento de esta se basa en informar la salida máxima dentro de una vecindad rectangular, por lo que además sirve para aumentar la no linealidad y mantener solo valores no negativos en la imagen.

En la tercera etapa se utiliza una función de agrupación conocida como *pooling* para modificar la salida de la capa y preservar las características dominantes. Esta función ayuda a reducir el tamaño espacial de la característica convolucionada, lo cual a su vez disminuye la potencia computacional

requerida para procesar los datos. Existen varias funciones de agrupación, entre las más utilizadas se encuentra *MaxPooling*, la cual se basa en regresar el valor máximo de la parte de la imagen cubierta por el *kernel*.

Finalmente, el *flattening* se encarga de convertir un mapa de características agrupadas en un vector de columna unidimensional. La salida aplanada se envía a una red neuronal de retroalimentación con *backpropagation* aplicada a cada iteración. A lo largo de una serie de épocas, el modelo puede identificar las características dominantes y las características de bajo nivel en las imágenes. Con esto, se puede utilizar *softmax* que entrega los valores de salida entre 0 y 1 para realizar una clasificación.

Los componentes descritos para una capa de una CNN típica se pueden observar en la figura 2.4, donde se ve un ejemplo de una capa totalmente conectada para la identificación de números manuscritos.

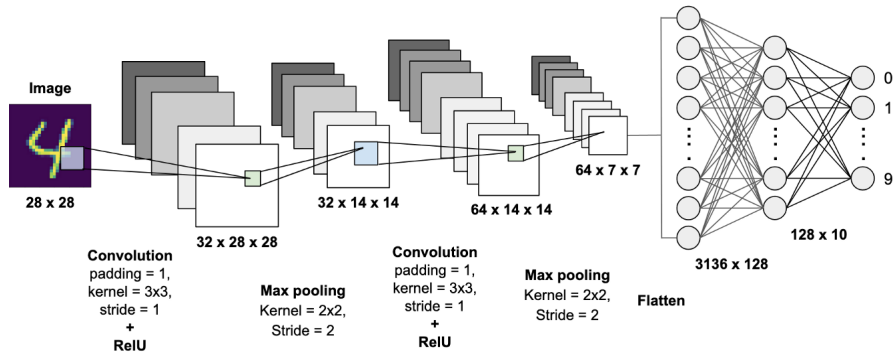


Figura 2.4. Capa totalmente conectada en una CNN [3].

2.3. YOLO

YOLO es una red neuronal convolucional (CNN) en donde todos los componentes para la detección de objetos se unifican en una sola red neuronal. En esta se enmarca la detección de objetos como un problema de regresión sobre cuadros delimitadores separados y con probabilidades de clases asociadas. Esto permite que la red sea capaz de razonar globalmente y predecir los cuadros delimitadores de todas las clases de forma simultánea [4].

En cuanto al tipo de aprendizaje automatizado de YOLO, este entra dentro de la categoría de aprendizaje supervisado, ya que requiere de anotaciones de las posiciones y clases de cada objeto que se quiera detectar para entrenar a la red.

2.3.1. Arquitectura

Tal y como se ve en la figura 2.5, la arquitectura de YOLO consiste en una red de detección con 24 capas convolucionales, en las cuales se extraen características de la imagen, mientras que las dos capas completamente conectadas que le siguen predicen las probabilidades y coordenadas de salida. Esta arquitectura tiene como base el modelo descrito por GoogLeNet para la clasificación de imágenes [4].

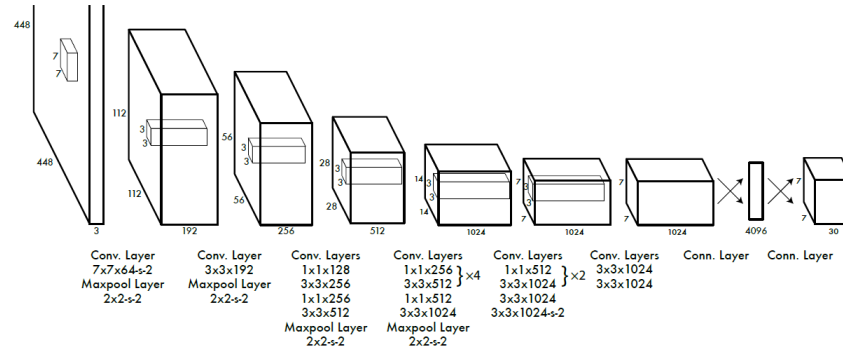


Figura 2.5. Arquitectura de YOLO [4].

2.3.2. Variantes de YOLO

La versión original de YOLO escrita por Joseph Redmon tiene diversas variantes desde su aparición, estas han mejorado con el tiempo incluso después de que su autor principal dejara la investigación. Dentro de los principales cambios que ha tenido esta tecnología se encuentran los conceptos *Bag of Freebies* y *Bag of Specials* que incluyen pasos de preprocesamiento y aumento de datos. También incluyen mejoras en temas como uso de GPU, rapidez y capacidad de aprendizaje de la red [20].

A continuación, se muestra la línea de tiempo con las diferentes versiones YOLO.

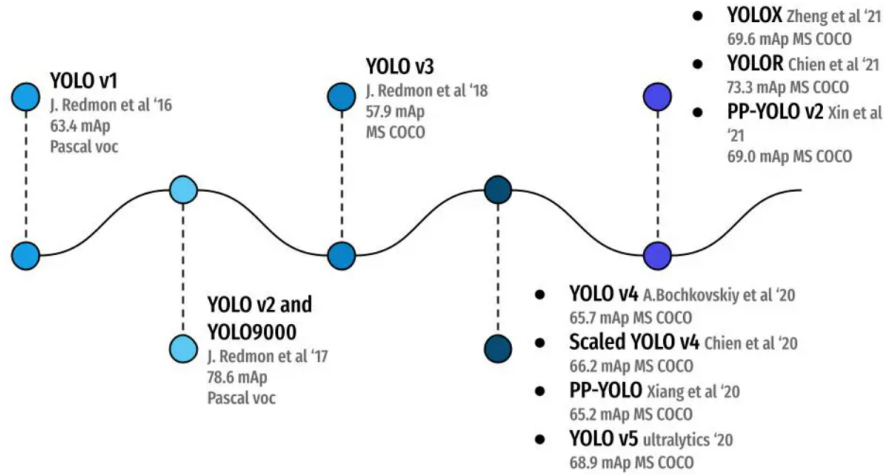


Figura 2.6. Historia de YOLO [5].

2.4. Generación de datos artificiales

Los métodos de aprendizaje profundo necesitan de grandes cantidades de datos de entrenamiento para cubrir su potencial. Por lo que la generación de datos sintéticos es de interés, ya que la creación de datos teóricamente puede tener una generación infinita de datos de entrenamiento con grandes variaciones, en donde el etiquetado viene de forma automática [21]. Adicionalmente, entrenar con datos sintéticos permite tener un control preciso sobre el renderizado de las imágenes y, por consiguiente, de las propiedades del *dataset*.

2.4.1. Brecha de dominio

Como se expresó previamente, las técnicas de generación sintética a partir de un motor gráfico ofrecen múltiples beneficios en cuanto a tiempo y costo. Sin embargo, este tipo de generaciones tienen un desafío a la hora de ser aplicados en la práctica. Esto debido al fenómeno llamado brecha de dominio entre los datos sintéticos y los datos reales. Tal y como se expresa en [22], los métodos entrenados por datos sintéticos evaluados en datos reales generalmente dan como resultado un rendimiento deteriorado.

Para adaptar el conocimiento del dominio a los datos del mundo real se han propuesto las siguientes técnicas:

- **Extensión con datos reales:** como se observa en el trabajo [23], al

utilizar un *dataset* artificial mezclado con datos reales se obtiene una mejora significativa. Estos enfoques tienen la ventaja de emplear datos del mismo dominio, ya que los objetos recortados son copias de imágenes reales y, como tales, se asemejan mucho a las características del mundo real. La principal limitación de estos enfoques es que requieren realizar el engorroso proceso de capturar imágenes de los objetos desde todos los puntos de vista posibles y enmascararlos.

- **Renderizado fotorrealista:** este se basa en utilizar renderización y composiciones de escenas realistas para superar la brecha de dominio mediante la síntesis de imágenes que se asemejan lo más posible al mundo real en cuanto a la imitación de luz, texturas y perspectivas. Aunque este enfoque muestra mejoras, su implementación implica una alta dificultad en realizar las composiciones de escenas y un uso significativo CPU/GPU para el renderizado [24, 25].
- **GANS:** trabajos como [26] utilizan las redes adversas generativas (GAN) para transformar imágenes de entornos simulados aleatorios en imágenes que parecen similares a las obtenidas de una imagen canónica no aleatoria. Sin embargo, tales enfoques implican una complejidad adicional, ya que son difíciles de diseñar y entrenar.
- **Adaptación del dominio:** tal y como se ve en los trabajos [27, 28], la adaptación de dominio (DA) aprovecha los datos anotados de un dominio de origen, en el que son abundantes, para entrenar a un clasificador que opere en un dominio de destino, en el que son escasos o incluso inexistentes. La principal desventaja de este enfoque es que requiere entrenamiento adicional con datos del mundo real.
- **Congelamiento de características:** se usan pesos congelados pre-entrenados en imágenes reales para permitir entrenar la parte restante de la arquitectura solo en imágenes sintéticas [22]. Si bien aprovecha las arquitecturas con extractores de características, requiere de la existencia de un preentrenamiento para su utilización.
- **Aleatorización de dominio:** como se observa en los trabajos [21, 29], se utilizan datos puramente sintéticos para la generación. Con esto, se busca que la variabilidad en la simulación sea lo suficientemente significativa para que los modelos entrenados en la simulación generalicen

el mundo real sin entrenamiento adicional. Aún cuando es una buena opción, ya que no depende de datos reales, su implementación requiere de realizar múltiples modelos sintéticos de objetos y fondos.

Capítulo 3

Metodología de trabajo

3.1. Investigación e hipótesis

En la primera etapa del proyecto se comienza con el estudio del estado del arte, que se enfoca en recopilar información sobre los conceptos relacionados con la investigación, tales como: el paquete de visión por computadora Perception, algoritmos de detección de objetos YOLO, técnicas de preprocesamiento de datos, técnicas de generación de datos, características fenotípicas de las colonias de bacterias y métricas de calidad orientadas al dominio de la biología.

Luego, sobre la base de los conocimientos recopilados, se plantea la hipótesis acerca de generar un *dataset* de colonias de bacterias sintético para la posterior implementación de algoritmos de tipo YOLO en la identificación y caracterización de una colonia de bacterias real con resultados adecuados.

3.2. Diseño

En el diseño se definen los tipos de bacterias objetivos con los que se trabajará en el modelo. De acuerdo con esta selección se recopilarán imágenes para la validación de datos y referencia del diseño de los objetos que se utilizarán en el generador. A partir de estos datos se deben generar las anotaciones adecuadas de forma manual para que posteriormente se empleen como datos de prueba. En esta etapa también se estipulan las métricas para evaluar la

calidad del conjunto de datos. Tal y como se mencionó en la solución preliminar, es importante que la cantidad de bacterias detectadas sea suficiente para considerarse como una muestra representativa de la colonia. Por lo que con base en [30], se define identificar al menos un 80 % de la población de bacterias.

Sobre la base de las imágenes recopiladas para las pruebas, se eligen algunas para utilizar de referencia en el diseño de los modelos 3D, escenarios y materiales. Luego, se realizan los programas de aleatoriedad y se configura el generador para los modelos, escenarios y materiales creados, es decir, variación en la posición de bacterias, color y tamaño en cada captura de datos. Posterior a esto, se seleccionan las versiones de YOLO que se implementarán y se realiza un programa que lleve a cabo la estandarización, limpieza, formato de las imágenes y etiquetas para probar los datos.

Finalmente, se realiza el entrenamiento de la red neuronal y se prueba con el conjunto de datos reales. A partir de estas pruebas se puede ajustar y calibrar la generación de los datos para obtener mejores resultados.

3.3. Experimentación y análisis de resultados

Luego del diseño y la implementación se tendrá una fase orientada a realizar pruebas utilizando los *datasets* generados. En ella se almacenarán los resultados para el posterior análisis, y se documentará cualquier información importante obtenida a partir de estos. Para validar el modelo se utilizarán las métricas planteadas en el diseño del experimento, considerando fundamental la distinción de formas y detección de una cantidad representativa de bacterias en una colonia.

Finalmente, luego de experimentar, se levantan conclusiones a partir de las pruebas realizadas, con la finalidad de verificar si estos respaldan o no la hipótesis. Una vez terminado, se redacta un informe que recabe la información de todas las etapas mencionadas anteriormente, incluyendo esta.

Capítulo 4

Solución propuesta

En este capítulo se explica la solución propuesta, el diseño, la implementación y los experimentos realizados. En cada uno de estos se muestran las herramientas empleadas.

4.1. Diseño

La solución consiste en utilizar el módulo de Perception Unity para generar un *dataset* sintético de bacterias por medio de un entorno de renderización. Para utilizar este módulo se deben generar todos los materiales necesarios, y realizar un diseño adecuado para el dominio objetivo.

Dentro de la solución se deben considerar técnicas que reduzcan la brecha de dominio. Entre estas se seleccionan el enfoque de extensión con datos reales y el enfoque de aleatorización de dominio.

Una vez obtenido el *dataset* sintético se utiliza la red neuronal YOLO para entrenar un detector y clasificador multiclase de bacterias. En este se realizan validaciones de los parámetros óptimos para tener un entrenamiento con tiempo y calidad razonables. Luego, se realiza una comparación entre las versiones de YOLO implementadas. Posteriormente, se realiza una búsqueda para mejorar la configuración del generador junto con la prueba e incorporación de aleatorizadores personalizados. Finalmente, con base en los experimentos anteriores, se utiliza la mejor configuración encontrada y se realizan las pruebas para validar el *dataset* sintético.

Cuando se tengan todos los resultados de los experimentos, con el *dataset* que entregue mayor calidad de detección, se comprobará si la hipótesis planteada es correcta.

4.1.1. Diseño de la solución

En la figura 4.1 se presenta un esquema que resume el flujo de la solución que se implementará.



Figura 4.1. Flujo de la solución propuesta.

Tal y como se observa en el flujo de la solución, previamente al entrenamiento de YOLO, se debe realizar el modelamiento, configuración y conversión de los datos. Cuando se obtienen los datos con el preprocesamiento realizado, se procede a entrenar la implementación de YOLO seleccionada para, posteriormente, comparar los resultados de los diversos experimentos.

4.1.2. Definición de bacterias y tipos de tinciones

Para diseñar los materiales que se utilizarán en la implementación del generador se debe definir a qué formas de bacterias estará orientada. Como también, qué tipo de tinción se utilizará para la validación en imágenes reales. Tal y como ve en la sección 2.1, las formas más comunes son los bastones, esferas y espirales, por lo que en esta instancia de investigación se limitará solo a estas tres formas.

En cuanto a las tinciones, para esta investigación se acotará a detectar e identificar bacterias obtenidas por técnicas de tinción diferencial GRAM, ya es uno de los métodos más conocidos y visualmente representativos para la detección.

4.1.3. Definición de métricas para los experimentos

Para modelos de tipo YOLO se suelen usar métricas basadas en la cantidad y confianza de los objetos detectados, ya sea en acertar en la clase o en la posición que se coloca el recuadro de predicción.

A continuación, con base en [31] se describen las métricas que se utilizarán en los experimentos:

- **Umbral de confianza:** es un valor que se define a la hora de realizar la detección para filtrar los resultados con poca confianza en las distintas clases del modelo.
- **Precisión:** es la relación entre el número de muestras positivas clasificadas correctamente y el total de muestras clasificadas como positivas (correcta o incorrectamente). Por lo que la precisión indica la exactitud del modelo en clasificar una muestra como positiva.
- **Recall:** es la relación entre muestras positivas correctamente clasificadas como positivas, y el número total de muestras positivas. Este indica la capacidad del modelo de detectar muestras positivas sin importar si se acierta o no.
- **Valor F1:** mide el equilibrio entre precisión y *recall*. Cuando el valor de F1 es alto, significa que la precisión y el *recall* tienen valores similares. En cambio, un valor F1 bajo significa que hay un mayor desequilibrio entre la precisión y el *recall*.
- **Umbral IoU:** el valor IoU (Intersection over Union) se calcula como la relación del área intersectada entre un objeto y su cuadro detectado dividido en la unión entre el objeto y el cuadro detectado. Por otro lado, el umbral IoU es la media de los IoU de los objetos detectados para un umbral específico.
- **mAP50:** el valor de mean Average Precision (mAP) entrega un resumen de qué tan bueno es el modelo de entrenamiento. Este se calcula tomando la media de la precisión obtenida por el área bajo la curva de precisión versus *recall* sobre todas las clases y/o umbrales globales de IoU. Mientras que en el mAP50 se calcula como la media del AP para todas las clases con umbral IoU de 0.5.
- **Matriz de confusión:** entrega un mapa que indica la probabilidad de confusión entre los valores predichos y los reales. Usualmente, se representa con la intensidad de colores, a mayor intensidad mayor es la probabilidad. Por otro lado, también sirve para ver las falsas alarmas en la misma clase, si el valor de confusión es bajo entre una clase y ella misma, es porque hay mayor cantidad de falsos positivos.

Se destaca que estas métricas se calcularán en un conjunto de testeo independiente de los conjuntos de entrenamiento y validación que se utilicen.

Esto debido a que un sistema que calcula sus métricas sobre *datasets* sintéticos o mixtos, tanto en entrenamiento como validación, pueden tener sesgos de aprendizaje que no determinen una real evaluación de la calidad.

4.2. Implementación

4.2.1. Recopilación de datos reales

La recopilación de datos reales representa un paso fundamental para la validación de la calidad de los datos generados por el simulador. Con esto, se recopilan un total de 129 imágenes reales de bacterias, las cuales no son suficientes para los conjuntos de entrenamiento, validación y testeo que se requieren para la experimentación. Por consiguiente, se realiza un proceso de aumento de los datos a partir de recortes en las imágenes. Para cada imagen se hacen recortes de 240x240 píxeles, lo cual asegura una amplificación de la imagen que simplifica el proceso de anotación, ya que hay menos bacterias por imágenes.

Luego de este paso se tienen 900 imágenes distintas, de las cuales se eliminan las que tienen información nula. Finalmente, queda un total de 225 imágenes adecuadas, estas se distribuyen de la siguiente forma: 100 imágenes de entrenamiento, 50 de validación y 75 de testeo. En cuanto al proceso de etiquetado, este se realizó de forma manual con las tecnologías de LabelImg [32] y Roboflow [33].

4.2.2. Creación de materiales

En la etapa de modelamiento se deben hacer los objetos, materiales, animaciones y fondos necesarios para la posterior implementación en Unity. Este debe estar guiado en las imágenes de referencia recopiladas, para así considerar variaciones realistas en las formas que pueda tener cada tipo de bacteria. La herramienta que se utiliza para hacer los modelos y las animaciones es Blender¹.

4.2.2.1. Objetos principales

Los objetos principales, también conocidos como *foreground objects*, son las bacterias. Estas se encontrarán en una capa intermedia del escenario gene-

¹Blender: programa dedicado al modelado, iluminación, renderizado, animación y creación de gráficos tridimensionales.

rado. Como se ve en la imagen 4.2, se realizaron 21 modelos de bacterias: 11 con forma de espiral, que serían espirilos y espiroquetas; cinco con forma de cilindro, correspondientes a bacilos; y cinco con forma de esfera, denominados cocos.

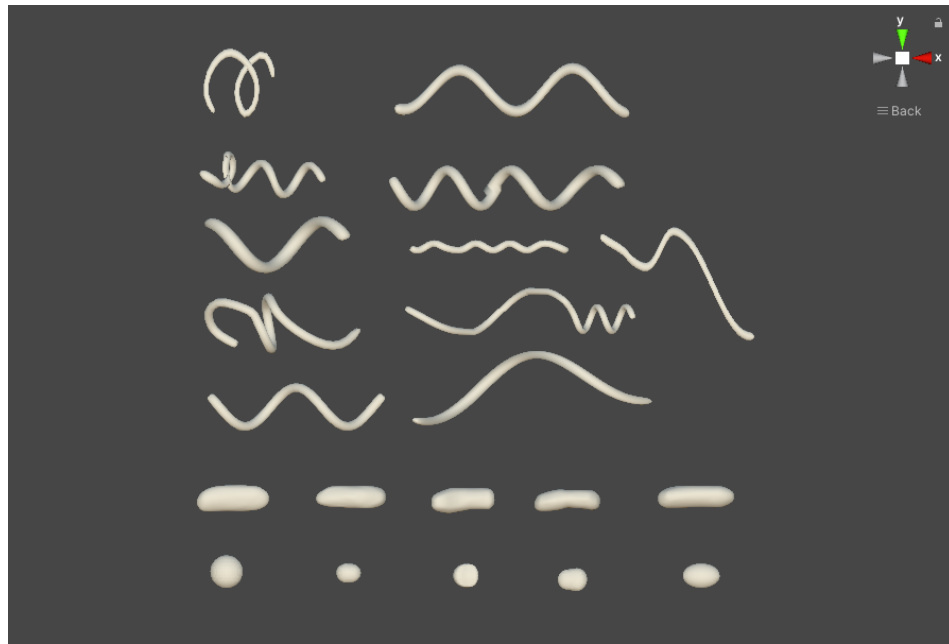


Figura 4.2. Diseño de objetos: bacterias con forma de espiral, cilíndrica y esférica.

Cada uno de estos modelos cuenta con una animación de 15 segundos en los que la bacteria se mueve en distintas direcciones y toma formas similares previas a la división celular. Por ejemplo, como se ve en la figura 4.3, en el caso del bacilo la animación pasa por el crecimiento bacteriano en donde se va alargando y achatando en el centro, pero nunca separándose.

La implementación en Unity considera esta animación de 15 segundos en 14 *frames* de poses. Por lo que se puede decir que se tiene un total de 294 poses distintas de bacterias para la generación de imágenes. Considerando una pose como una variación del modelo de la bacteria en cuanto a deformaciones en su estructura.

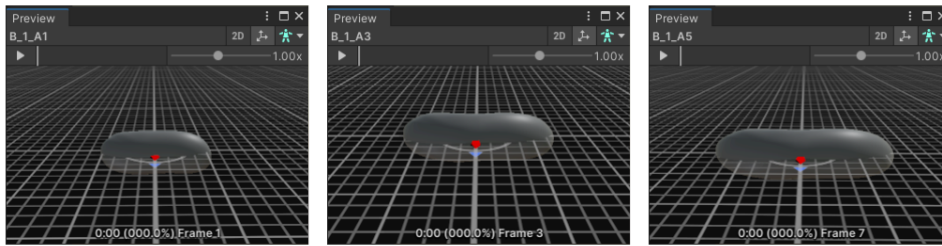


Figura 4.3. Modelo de bacilo en tres poses distintas de una animación.

En cuanto a las texturas, se considera la manera en cómo se ven las bacterias bajo el microscopio. Comúnmente las bacterias, al ser expuestas en un vidrio con una luz artificial, la fuente de luz es única. Además, varios tipos de bacterias tienen una composición transparente que se logra observar con la tinción. Como se observa en la figura 4.4, se diseñó un material único que reacciona de forma pareja a la luz. Por otro lado, este material se hizo con un nivel de translucidez medio para que la luz pueda traspasarlo, y así simular la transparencia de las bacterias.

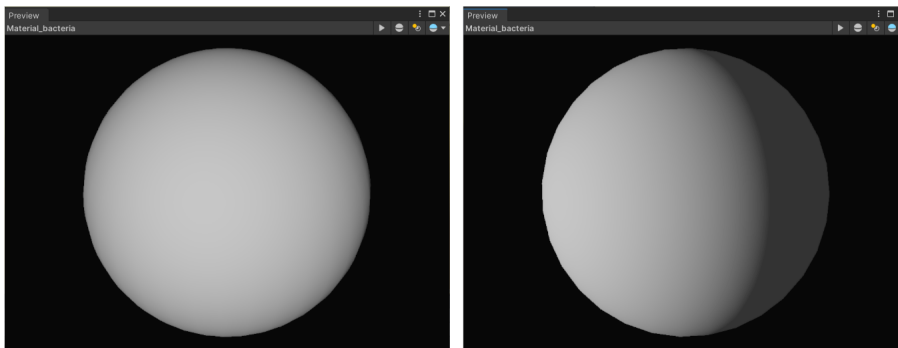


Figura 4.4. Muestra del material diseñado para las bacterias.

4.2.2.2. Objetos de fondo

Para los objetos de fondo o *background objects*, se usaron formas geométricas comunes, como es el caso de los cilindros, esferas y cubos. Todos estos vienen incorporados junto con el paquete Perception.

En cuanto a las texturas de los objetos de fondo, estas dependen de la configuración. En una primera etapa se utilizan las texturas de imágenes reales de frutas y verduras, con la finalidad de crear un entorno ruidoso para el entrenamiento. Posteriormente, se realizarán pruebas de variación en los

colores, texturas y posición para seleccionar el fondo que mejor resultado entregue en el entrenamiento.

A continuación, en la figura 4.5 se pueden observar las formas y un ejemplo de textura que se utilizará en los objetos de fondo. La diferencia entre formas de un mismo tipo es que los bordes pasan a ser más suaves y circulares.

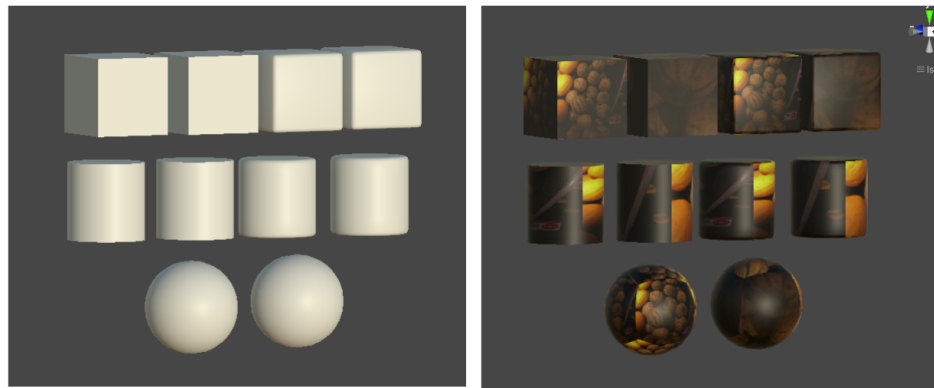


Figura 4.5. Objetos de fondo y textura de ejemplo.

4.2.3. Implementación del generador

El generador se estructura de tres componentes principales: escenario, cámara y fuente de iluminación. Estos se ubican en la simulación, como se observa en la figura 4.6, en donde primero se encuentra el escenario con los objetos, luego la cámara y, finalmente, la iluminación, todas apuntando desde el centro hacia el eje z.

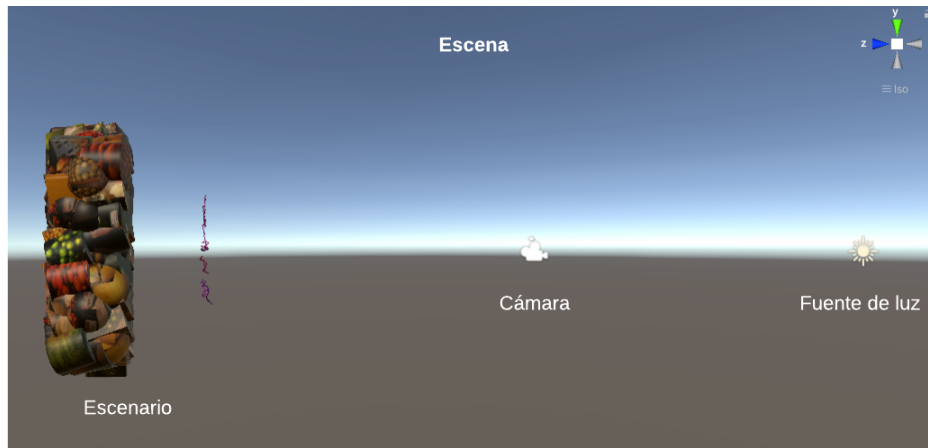


Figura 4.6. Componentes en la escena del simulador.

A continuación, se explica a detalle la configuración utilizada para cada uno de estos componentes.

4.2.3.1. Escenario

El escenario tiene las responsabilidades de controlar la ejecución de la simulación, definir una lista de aleatorizadores y de constantes modificables de forma externa. Se busca que cada iteración de un escenario pretenda encapsular la ejecución de un entorno con condiciones aleatorias únicas.

En cuanto a los parámetros del escenario, se encuentran las siguientes entradas:

- **Random seed:** el *random seed* da como resultado un *sampler*, esta es una clase que genera de forma determinista valores flotantes aleatorios a partir de distribuciones de probabilidad limitadas.
- **Iteración de inicio:** índice de la primera iteración a ejecutar.
- **Cantidad de iteraciones:** número de iteraciones a ejecutar.
- **Frames por iteración:** número de *frames* a ejecutar por iteración. Este parámetro se utiliza para mantener los objetos de fondo fijos mientras se ejecuta una animación.

Por otro lado, un escenario se compone de aleatorizadores. Estos manejan actividades de aleatorización específicas para realizar en las simulaciones. Como por ejemplo: generar objetos, cambiar posiciones, variar colores, entre

otros. Mientras que los *randomization tag* son el mecanismo para definir un subconjunto de *GameObjects* para saber qué objetos deben variar por los aleatorizadores. Además, dentro de estos se pueden definir valores flotantes aleatorios acotados mediante el muestreo de distribución de probabilidad. Estos son acotados, ya que cada *sampler* genera valores dentro de un rango definido por un valor mínimo y máximo.

Los aleatorizadores se deben colocar en el orden que serán ejecutados: los de arriba se ejecutan primero de los que se colocan por abajo en la lista. Perception ofrece un conjunto de aleatorizadores que sirven para tener una configuración base, pero también es posible programar aleatorizadores personalizados para tareas específicas de un dominio.

Para este caso, se desarrollaron una serie de aleatorizadores personalizados enfocados en la microbiología. Estos son aleatorizadores de posición basada en la agrupación y color de la iluminación.

4.2.3.2. Cámara

La cámara contiene las configuraciones para guardar los resultados en la memoria del dispositivo, propiedades de calendarización de captura y los *labels* que se recolectarán. En esta ocasión se utiliza un *label* de tipo *BoundingBox2DLabeler*, ya que es el requerido para los algoritmos YOLO. Mientras que en los objetos que se quiere capturar se coloca un componente de *labeling*, en el cual se especifican los *labels* que pertenecen al objeto específico. Todo esto junto con un archivo .asset de configuración de *labels* en donde se especifican los ID de cada uno de los objetos.

4.2.3.3. Fuente de luz

En la fuente de luz se puede modificar parámetros de tipo, color, modo, intensidad, tipo de sombras, tipo de renderización y máscara de ocultación de la luz. En este se puede colocar un *randomization tag* para cambiar factores como la latitud, día del año y hora del día que representará la luz en las simulaciones.

4.2.4. Conversión de formato

La ejecución de la simulación entrega una carpeta llamada RGB y otra llamada Database. La primera contiene las imágenes generadas y la segunda

archivos .json para las anotaciones y la información sobre cada *label* indicado en la configuración.

Los archivos de captura recopilan los *labels* designados para cada modelo de la simulación, y para cada instancia de captura de imagen. Los datos que se guardan en el archivo sobre los *bounding box* son los siguientes:

- **label__id:** ID correspondiente a la clase.
- **label__name:** nombre de la clase del objeto.
- **instance__id:** ID de la instancia.
- **x:** posición de inicio del *bounding box* en el eje x.
- **y:** posición de inicio del *bounding box* en el eje y.
- **width:** ancho del *bounding box*.
- **height:** largo del *bounding box*.

Se requiere de una conversión para utilizar las anotaciones generadas en el entrenamiento de YOLO. Para esto se implementan dos códigos, uno para YOLOv4 y otro para YOLOv5. Estos se encuentran en los anexos A.1 y A.3 respectivamente. A su vez, Roboflow también ofrece compatibilidad con las anotaciones generadas por el simulador, por lo que se puede realizar la conversión en ella.

4.2.5. Implementación de YOLOv4

En esta sección se verá la implementación de YOLOv4, partiendo desde el preprocesamiento necesario para luego definir los parámetros de entrada que se necesitan en el entrenamiento. La implementación de YOLOv4 que se utilizará es la de Darknet, esta se encuentra en el repositorio de github [34].

4.2.5.1. Preprocesamiento

YOLOv4 requiere de archivos que le indiquen las clases, ubicaciones y distribución de los datos en los conjuntos de entrenamiento y validación. Para esto se deben generar los siguientes archivos:

- **classes.names:** archivo con los nombres de las clases que se detectarán.

- **train.txt:** archivo con la ruta completa de las imágenes que serán para entrenar.
- **test.txt:** archivo con la ruta completa de las imágenes que serán para validar.
- **data.image.txt:** archivo con el número de clases, direcciones de los archivos train.txt, test.txt, classes.txt y ubicación de dónde se guardarán los datos de *backup*.

En la carpeta donde se ubica la imagen es necesario que se encuentre el archivo .txt de anotaciones con el mismo nombre de la imagen. Se implementó un código para automatizar la creación de estos archivos. Este se encuentra en el anexo A.2.

4.2.5.2. Entrenamiento

Para el entrenamiento de YOLOv4 se define la ubicación del archivo imagen_data.data, la ubicación del archivo .cfg y los pesos a utilizar en el modelo. El archivo de configuración tiene las características de la arquitectura que se utilizará para el entrenamiento. Este contiene principalmente las siguientes propiedades:

- **batch:** cantidad de imágenes anotadas que se utilizan en el *forward pass* para computar el gradiente y actualizar los pesos con *backpropagation*.
- **subdivisions:** el *batch* es subdividido en estos bloques, las imágenes de un bloque son ejecutadas en paralelo en la GPU.
- **width:** las imágenes serán cambiadas de tamaño a este ancho.
- **height:** las imágenes serán cambiadas de tamaño a este alto.
- **steps:** es la cantidad de iteraciones en donde se hace un punto de control para aplicar la escala. Su valor recomendable es de 80 % y 90 % del *max_batches*.
- **max_batches:** el entrenamiento se procesará para este número de iteraciones (lotes). Generalmente, se calcula como el número de clases por 2000.
- **filters:** es el número de filtros que utilizan las capas de YOLO. Se suele calcular como (número de clases + 5) * B, con B = 3.

- **classes:** número de clases que tiene la instancia a entrenar.
- **cfg:** dirección hacia el archivo de configuración.
- **weights:** dirección hacia el archivo de pesos preentrenados, el recomendado es yolov4.conv.137. Al usar este archivo para entrenar un modelo propio se obtienen patrones de bajo nivel: líneas y círculos. Como también patrones más complejos del *dataset* MSCOCO [35]. Por ello, se utiliza ese conocimiento para adaptar el modelo a una tarea propia.

4.2.6. Implementación de YOLOv5

Para la confección de YOLOv5 se usa la implementación de Ultralytics [36]. Esta ofrece compatibilidad con la plataforma WandB que entrega gráficas para los experimentos. En cuanto al modelo, se utiliza YOLOv5s, que es un modelo previamente entrenado desde PyTorch hub, este es rápido y ligero para experimentos de tamaño pequeño a medio.

4.2.6.1. Preprocesamiento

YOLOv5 funciona con un único archivo .yaml. Pero también requiere de una estructuración de carpetas específica, como se observa en la figura 4.7. Se deben separar las imágenes de los *labels* y los conjuntos de datos de *train*, *valid* y *test* en diferentes carpetas.

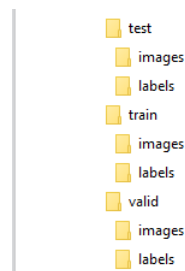


Figura 4.7. Estructura de carpetas para YOLOv5.

Dado esto, para el preprocesamiento solo se requiere distribuir los archivos en entrenamiento, validación y testeo. Y luego, indicar la posición relativa de estas carpetas en el archivo .yaml. Como también, el número de clases y sus ID. En el anexo A.3 se encuentra el código que automatiza la conversión de formato, estructuración y creación del archivo .yaml.

4.2.6.2. Entrenamiento

Para el entrenamiento de YOLOv5 se definen las siguientes entradas:

- **img:** es la resolución de la imagen que se utilizará en el entrenamiento.
- **batch:** cantidad de imágenes a las que se le extraen las características de manera concurrente durante el entrenamiento.
- **epoch:** número de veces que pasa el *dataset* de entrenamiento completo por la red.
- **data:** ubicación del archivo `.yaml`.
- **weight:** pesos del modelo preentrenado a utilizar, en este caso se seleccionó el modelo `yolov5s`.
- **cache:** sirve para guardar las imágenes que se utilizarán para el entrenamiento en el caché y así aumentar la rapidez.
- **patience:** sirve para que la red no finalice el entrenamiento cuando detecte que las métricas no varían en un número dado de iteraciones.

4.3. Experimentación

Los experimentos que se realizaron fueron de cuatro tipos, el primero para seleccionar parámetros adecuados en el entrenamiento de YOLO; el segundo para evaluar el rendimiento de las versiones de YOLOv4 y YOLOv5; en el tercero, se varían los distintos aleatorizadores y sus valores para tener una generación que se adecúe más al dominio específico; y en el cuarto, se mide la calidad del *dataset* sintético en comparación a un *dataset* real y otro mixto. Para todas estas pruebas se utilizó Paperspace Gradient. La instancia utilizada se constituye de una GPU NVIDIA RTX A4000 y una memoria interna de 45GB RAM.

4.3.1. Experimentos de sensibilidad de parámetros de entrenamiento

Para validar la implementación de YOLOv5 se realiza un conjunto de experimentos con variaciones en épocas, tamaño de *batch* y tamaño de imágenes. En estas pruebas se utiliza el *dataset* con la configuración base definida en la sección 4.3.3.

En los siguientes experimentos se varía uno de los parámetros, mientras los demás se dejan constantes con los siguientes valores:

- **Épocas:** 300.
- **Tamaño de *batch*:** 32.
- **Tamaño de imagen:** 416 píxeles.

4.3.1.1. Épocas

En la tabla 4.1 se muestra el tiempo de ejecución, la precisión (P), el *recall* (R) y el mAP50. Como se vio en la sección 4.1.3, la precisión indica qué tan bueno es el modelo para detectar aciertos positivos, el *recall* es la capacidad de detección de positivos independiente de si son o no correctos y el mAP50 es un resumen de la calidad del entrenamiento para umbral IoU de 50 %. En este caso se varían las épocas en las cantidades de 100, 200, 300, 400 y 1000.

Época	Tiempo	P	R	mAP50
100	0.26 (horas)	0.676	0.477	0.491
200	0.51 (horas)	0.611	0.514	0.508
300	0.76 (horas)	0.586	0.507	0.502
400	0.97 (horas)	0.639	0.45	0.462
1000	2.47 (horas)	0.614	0.467	0.483

Tabla 4.1. Comparación entre cantidad de épocas.

4.3.1.2. Tamaño de *batch*

En este experimento se utilizan variaciones del tamaño de *batch* entre los siguientes valores: 8, 16, 32 y 64.

A continuación, se ve la tabla resultante con el tamaño de *batch*, tiempo de ejecución, precisión, *recall* y mAP50.

<i>Batch</i>	Tiempo	P	R	mAP50
8	1.41 (horas)	0.564	0.509	0.474
16	0.90 (horas)	0.57	0.508	0.463
32	0.76 (horas)	0.586	0.507	0.502
64	0.69 (horas)	0.591	0.515	0.479

Tabla 4.2. Comparación entre tamaños de *batch*.

4.3.1.3. Tamaño de imágenes

Para el tamaño de imagen se realizan variaciones con los valores de 416 y 640 píxeles.

A continuación, se muestra la tabla resultante de estas pruebas con el tamaño de imagen, tiempo de ejecución, precisión, *recall* y mAP50.

Tamaño	Tiempo	P	R	mAP50
416 (píxeles)	0.76 (horas)	0.586	0.507	0.502
640 (píxeles)	1.24 (horas)	0.556	0.443	0.408

Tabla 4.3. Comparación entre tamaños de imágenes.

4.3.2. Comparación de versiones YOLO

En este experimento se busca comparar la diferencia entre los resultados de entrenamiento para las implementaciones realizadas de YOLOv4 y YOLOv5.

Los parámetros utilizados en esta comparación son los siguientes:

- **Épocas:** 100.
- **Tamaño de *batch*:** 32.
- **Tamaño de imagen:** 416 píxeles.

A continuación, se ve la tabla resultante con el tiempo de ejecución, precisión, *recall* y mAP50 para las versiones de YOLOv4 y YOLOv5.

	Tiempo	P	R	mAP50
YOLOv4	6.49 (horas)	0.14	0.50	0.298
YOLOv5	0.26 (horas)	0.67	0.47	0.491

Tabla 4.4. Comparación entre YOLOv4 y YOLOv5.

4.3.3. Pruebas de aleatorización de dominio

El objetivo de estos experimentos es determinar los parámetros de configuración para la generación de datos que mejoran el entrenamiento. Además, se busca ver cómo cambia el resultado del entrenamiento al añadir aleatorizadores personalizados en el generador. Para esto, primero se define una configuración base y, luego, se varían los aleatorizadores junto con sus parámetros asociados.

Los parámetros de entrenamiento que se utiliza en la red neuronal son los siguientes:

- **Épocas:** 200.
- **Tamaño de *batch*:** 32.
- **Tamaño de imagen:** 416 píxeles.
- **Tamaño del *dataset*:** 2100 imágenes.

A continuación, se describe la configuración del escenario:

- ***Random seed*:** semilla de randomización estándar con valor 539662031.
- **Iteración de inicio:** la iteración se inicia en 0.
- **Cantidad de iteraciones:** 700 por cada forma bacteriana.
- ***Frames* por iteración:** 1 *frame* por iteración. Esto debido a que las poses se configuraron para que en cada animación elegida el escenario cambie completamente.

En cuanto a los aleatorizadores entregados por el módulo de Perception, se utilizan los siguientes:

- **Aleatorización de posición de objetos de fondo:** los objetos de fondo se colocan de forma aleatoria a una profundidad mayor a la de la capa principal.
- **Aleatorización de posición de objetos principales:** los objetos principales tienen una probabilidad uniforme de aparecer en cualquier parte de la escena. Dependiendo de la distancia que se quiera tener entre los objetos, aparecerán más o menos modelos en la capa principal. Para esta instancia se utiliza una distancia entre objetos de 0.8, lo que da un total de 10 bacterias por imagen.
- **Aleatorización de texturas:** la aleatorización de texturas se utiliza solo en la capa de fondo. Esta varía de forma uniforme entre las texturas predefinidas, generando ruido en las imágenes.
- **Aleatorización de color de objetos de fondo:** en la aleatorización de color de los objetos de fondo se selecciona la gama completa del formato HSV.

- **Aleatorización de color de objetos principales:** la aleatorización de color en los objetos principales se encarga de pintar la textura base que se utiliza en las bacterias. Se seleccionó un rango de colores basado en el formato HSV, en el cual se toma una gama de colores orientada a la tinción GRAM con alta variación en saturación y brillo.
- **Aleatorización de rotación de objetos de fondo:** la aleatorización de objetos de fondo tiene rotaciones aleatorias uniformes desde 0° hasta 360° para todos los ejes.
- **Aleatorización de rotación de objetos principales:** se debe tener una consideración especial al designar los valores de rotación de las bacterias, ya que al tener posiciones de giro verticales de 90° pueden tomar una forma confusa. Adicionalmente, se contradice con la orientación que tienen las bacterias en las muestras, las que se ubican recostadas horizontalmente. Con esto en consideración, se seleccionan los siguientes rangos: en el eje x desde 0° hasta 40° , en el eje y desde 0° hasta 40° y en el eje z desde 0° hasta 360° .
- **Aleatorización de animación:** las animaciones para cada modelo fueron repartidas en 14 poses diferentes. Por lo que en cada instancia de escenario se selecciona una de esas poses con probabilidad uniforme.
- **Aleatorización de iluminación:** se utiliza un rango de horas entre 0 y 24, para los 365 días del año y con una latitud de desde 0° hasta 90° , considerando -90° como el polo sur y 90° como el polo norte.
- **Aleatorización de HUE:** afecta al fondo y cambia sus tonalidades de colores según el formato HUE entre los rangos de -180 a 180.

A su vez, se implementaron una serie de aleatorizadores personalizados para mejorar la calidad del generador de datos. Estos fueron diseñados considerando un enfoque hacia el dominio de la microbiología.

A continuación, se listan los aleatorizadores personalizados que se implementaron para el generador.

- **Aleatorizador de posición basada en la agrupación:** los objetos principales que se designen a este aleatorizador tienen una probabilidad del 80 % de tener una división. Esto quiere decir, que la posición del siguiente objeto principal será en una ubicación adyacente de la bacteria que se dividió. En este aleatorizador también se define la distancia

entre bacterias cuando no se dividan junto con su rango y profundidad del área de aparición. El código correspondiente a este aleatorizador se encuentra en el anexo A.4.1.

- **Aleatorizador de color e intensidad de iluminación:** la iluminación de la escena varía su intensidad y color con el que ilumina entre valores desde 0 hasta 1, la selección de rango de colores se basa en el formato RGBA. El código correspondiente a este aleatorizador se encuentra en el anexo A.4.2.

Dentro de los experimentos a realizar se encuentra la distancia y agrupación de bacterias, la variación de fondos, los parámetros de los modificadores y aleatorizadores personalizados. Todos estos se realizarán con la configuración definida previamente.

4.3.3.1. Distancia de bacterias y porcentaje de agrupación

Para determinar cómo afectan los parámetros de distancia y agrupación en el entrenamiento se realiza el siguiente proceso: en primer lugar, la distancia se varía en valores cercanos a 0.8, el cual entrega 10 bacterias por imagen, en segundo lugar, para la agrupación se configura un 80% de probabilidad, ya que este valor se asemeja al nivel de oclusión y aglomeración entre las bacterias que se encuentran en colonias reales.

A continuación, en la figura 4.8 se muestra cómo varía la calidad del entrenamiento para valores de distancias 0.4, 0.5, 0.6 y 0.8 con agrupaciones de 0% y 80%.

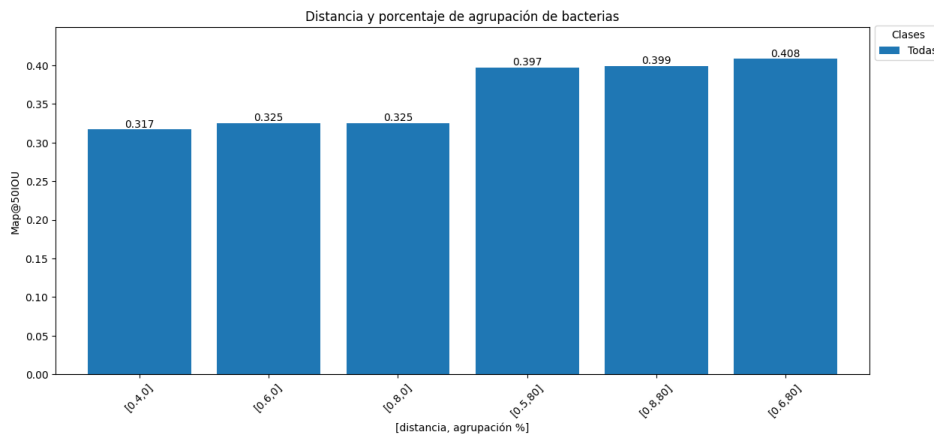


Figura 4.8. Gráfica de distancia y porcentaje de agrupación versus mAP50 para todas las formas.

4.3.3.2. Aleatorización de fondos

Para seleccionar el fondo que entregue mejor calidad de entrenamiento se probarán diversas formaciones de los aleatorizadores de color, texture y HUE. Un ejemplo con muestras de los *datasets* que poseen estos fondos se puede observar en la figura 4.9.

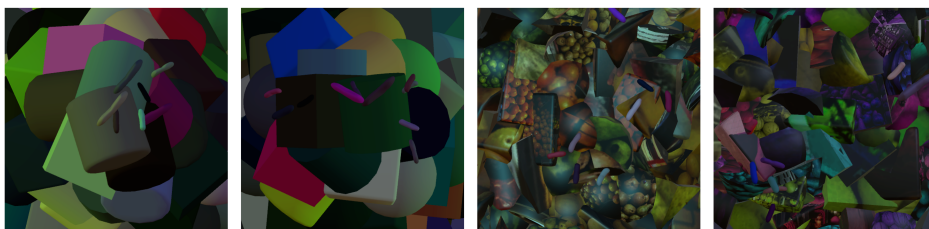


Figura 4.9. Muestra de los fondos de la experimentación para la forma de los bacilos: 1. fondo solo con variación de color; 2. fondo con variación de color y HUE; 3. fondo con variación de texturas; 4. fondo con variación de texturas y HUE.

A continuación, se muestra la gráfica resultante de la variación en los fondos del generador que entrega el mAP50 para todas las formas del modelo.

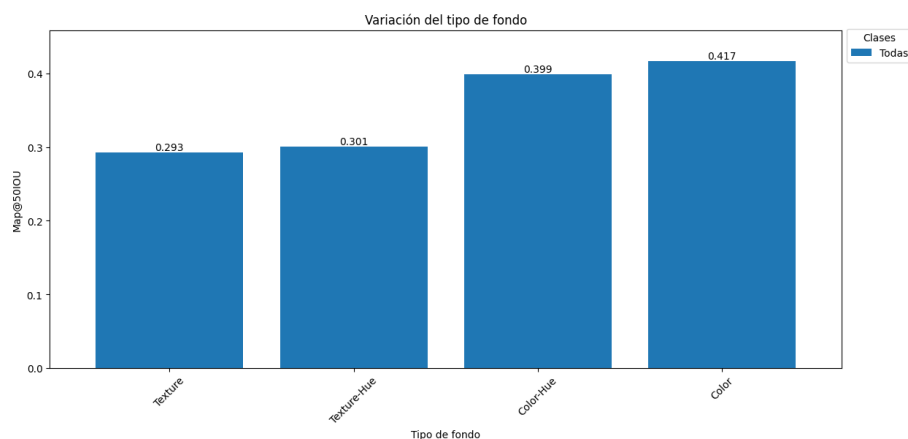


Figura 4.10. Gráfica de comparación de los tipos de fondos versus mAP50 para todas las formas.

4.3.3.3. Modificadores

Se seleccionaron diversos modificadores que pueden aumentar la capacidad de resiliencia ante desperfectos en la toma de muestras, como por ejemplo los cambios de *zoom* y desenfoques al deslizar el microscopio. Para comprobar cuáles influyen positivamente en el entrenamiento se realizan variaciones independientes de cada parámetro.

La configuración base que se utiliza en las pruebas es la siguiente:

- **Noise:** aplica ruido con valor aleatorio en las imágenes, los valores definidos son desde 0 % hasta 3 % de los píxeles de la imagen.
- **Blur:** aplica desenfoque gaussiano con valor aleatorio en las imágenes, los valores definidos son desde 0 hasta 3 píxeles.
- **Crop:** aplica un recorte con valor aleatorio en las imágenes, los valores definidos son desde 0 % hasta 30 %.

Además de estos modificadores, se verificará la influencia del aleatorizador de color e intensidad de la iluminación denominado *light color*, este se define previamente en la sección 4.3.3.

A continuación, se muestra la gráfica de la comparación del rendimiento de los modificadores previamente descritos que entrega el mAP50 para todas las formas del modelo.

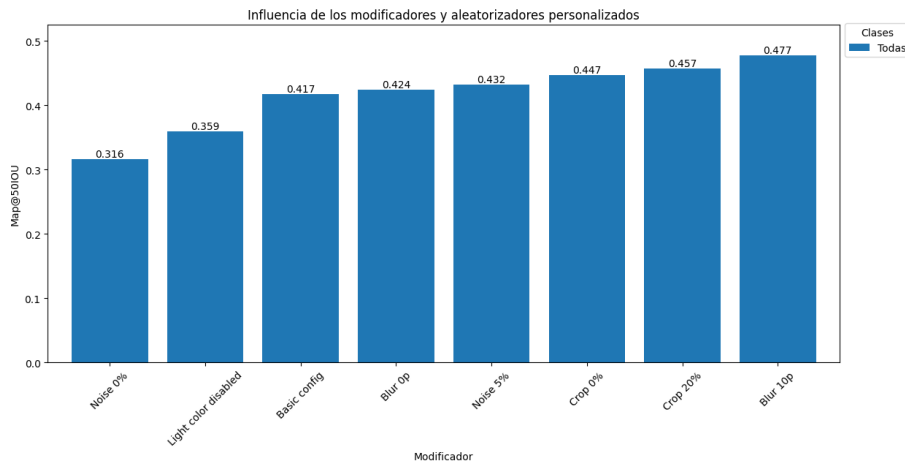


Figura 4.11. Gráfica de comparación de los modificadores versus mAP50 para todas las formas.

4.3.4. Validación de los *datasets*

A continuación, se realizan los experimentos que tienen como finalidad medir la calidad del *dataset* generado comparándolo con un *dataset* real y otro mixto.

Para los experimentos de los *datasets* generados se utiliza YOLOv5 con 300 épocas, tamaño de *batch* 32 y 416 píxeles de tamaño de imagen. Esto debido a que en las secciones 5.1 y 5.2 se observa que es la que tiene mejor

relación entre tiempo de ejecución y calidad. Mientras que para la generación se utiliza distancia 0.8, agrupación de 80 %, fondo con variación de colores y modificadores *noise*, *blur* y *crop* con valores 5 %, 5p y 20 % respectivamente.

En estas pruebas se realiza una comparación entre los *datasets* real, sintético y mixto. Para validar los entrenamientos se emplea un conjunto de testeo con 75 imágenes reales. El propósito de estas es dar una métrica realista con datos que no se han visto en el entrenamiento ni en la validación para tener una evaluación libre de sesgos en los datos.

Seguidamente, se describen los *datasets* utilizados en estas pruebas y su tabla resultante:

- **Dataset real:** el *dataset* real se compone de 150 datos etiquetados manualmente y su entrenamiento demoró 0.1 horas, esto debido a que es un *dataset* de tamaño pequeño.

A continuación, se muestra la tabla resultante con las clases, número de imágenes, instancias de la clase, precisión, *recall* y mAP50 para el *dataset* real.

Clase	Imágenes	Instancias	P	R	mAP50
Todas	75	760	0.744	0.662	0.697
Bacilo	75	399	0.759	0.671	0.699
Coco	75	215	0.706	0.684	0.673
Espirilo	75	146	0.767	0.632	0.720

Tabla 4.5. Tabla de resultados del *dataset* real.

- **Dataset sintético:** el *dataset* sintético de este experimento se compone de 9000 datos generados. Una muestra de los datos se puede encontrar en la figura B.1. En estos se separan las bacterias por imagen, es decir, se tienen 3000 imágenes de cada tipo de bacteria en forma individual. La razón de esto viene por las recomendaciones realizadas por [?], en donde señala que el mínimo recomendable para los entrenamientos es de 1500 imágenes por cada clase. El entrenamiento de este *dataset* demoró 1.91 horas.

A continuación, se muestra la tabla resultante con las clases, número de imágenes, instancias de la clase, precisión, *recall* y mAP50 para el *dataset* sintético.

Clase	Imágenes	Instancias	P	R	mAP50
Todas	75	760	0.621	0.51	0.506
Bacilo	75	399	0.679	0.541	0.54
Coco	75	215	0.566	0.297	0.3
Espirilo	75	146	0.618	0.692	0.677

Tabla 4.6. Tabla de resultados del *dataset* sintético.

- **Dataset mixto:** se incorporan 150 datos reales etiquetados de forma manual al *dataset* sintético, estos se dispersaron de forma aleatoria en los conjuntos de entrenamiento y validación, en un ratio 80 % para entrenamiento y 20 % de validación. En cuanto al tiempo de entrenamiento, este demoró 1.92 horas.

A continuación, se muestra las tablas resultantes con las clases, número de imágenes, instancias de la clase, precisión, *recall* y mAP50 para el *dataset* mixto.

Clase	Imágenes	Instancias	P	R	mAP50
Todas	75	760	0.78	0.676	0.743
Bacilo	75	399	0.733	0.551	0.613
Coco	75	215	0.742	0.724	0.782
Espirilo	75	146	0.866	0.754	0.833

Tabla 4.7. Tabla de resultados del *dataset* mixto.

En el anexo B se muestran las gráficas obtenidas en la validación. Estas se componen de curvas de precisión versus confianza, precisión versus *recall*, *recall* versus confianza, F1 versus confianza y matrices de confusión.

Capítulo 5

Análisis de resultados

En esta sección se realiza un análisis de los resultados obtenidos en los experimentos realizados.

5.1. Análisis de sensibilidad de parámetros de entrenamiento

En el análisis de sensibilidad de los parámetros de entrenamiento se observará el valor mAP50 para determinar la calidad del *dataset*, ya que esta métrica entrega un resumen de qué tan bueno es un modelo de entrenamiento.

- **Número de épocas:** de la tabla 4.1 se observa que el mAP tiene su valor máximo en la época 200, y luego va decayendo, hasta que en la época 1000 se mantiene en 0.483. Usualmente, el comportamiento que se tiene es que a mayor época la calidad del entrenamiento aumente, pero al utilizar datos sintéticos para entrenar y validar el modelo, este puede mejorar sus valores hacia una orientación distinta que la esperada. Por otro lado, se observa una relación directa en la cantidad de épocas y el tiempo de ejecución.
- **Tamaño de *Batch*:** de la tabla 4.2 se puede decir que el tiempo de ejecución disminuye cuando el *batch* es mayor, esto ocurre porque el modelo optimiza mejor sus recursos al procesar más datos en paralelo. Si bien comúnmente se tiene que con menor *batch* se obtiene más

calidad, esto no ocurre necesariamente para los casos puramente sintéticos. Por otro lado, si se compara la precisión y el *recall* de los *batch* 8 y 64, se tiene que con un bajo valor de *batch* la precisión aumenta y el *recall* disminuye, mientras que con un *batch* alto ocurre lo contrario, la precisión disminuye y el *recall* aumenta.

- **Tamaño de imagen:** de la tabla 4.3 se observa que el mAP disminuye a mayor tamaño y el tiempo de ejecución aumenta. Debido a que al tener mayor información que revisar la red se demora más en el entrenamiento. Además, la disminución del mAP al aumentar la resolución se puede justificar considerando el tamaño que tienen las imágenes de prueba son más cercanas a 416 píxeles.

Tras estos experimentos, se determina que la mejor configuración con base en la relación entre el tiempo y la calidad del entrenamiento es la combinación con 200 épocas, *batch* de tamaño 32, y tamaño de imagen de 416 píxeles.

5.2. Análisis de comparación de versiones YOLO

A continuación, se realiza el análisis para las versiones YOLOv4 y YOLOv5. En el cual se tendrá en consideración el tiempo que demoran en concretar 100 épocas y su calidad de detección para las métricas de *recall*, precisión y mAP50.

Como se ve en la tabla 4.4, el tiempo que se demora YOLOv4 en completar 100 épocas es de aproximadamente 6 horas más que para YOLOv5. Mientras que en cuanto a la calidad del entrenamiento, se tiene que YOLOv4 tiene una precisión de 0.14 y un *recall* de 0.50. Esto quiere decir que el modelo detecta varios objetos, pero la tasa de acierto en esas predicciones es baja. Mientras que en YOLOv5 se tiene una relación más pareja entre la precisión y el *recall*.

Por otro lado, si se comparan los valores de mAP50, YOLOv4 es aproximadamente 0.193 menor que el de YOLOv5. En conclusión, tanto en tiempo como en calidad de entrenamiento, YOLOv5 funciona mejor para los experimentos con el *dataset* sintético. Dado esto, se utilizará YOLOv5 en las pruebas de calidad de los *datasets* generados.

5.3. Análisis de técnicas de aleatorización de dominio

A continuación, se analizan los resultados obtenidos en los experimentos de las técnicas de aleatorización de dominio.

- **Distancia bacteriana y porcentaje de agrupación:** como se observa en la gráfica 4.8, la mejor distancia sin agrupación es con 0.8. Mientras que con agrupación la mejor es con 0.6. Además, los entrenamientos con agrupación entregan una mejora con respecto a los que no tienen, esto se debe porque la agrupación permite oclusión entre bacterias y posiciones similares a las que tendrían después de una división.
- **Aleatorización de fondos:** como se observa en la gráfica 4.10 se tiene que los fondos de color sin las texturas entregan mejores resultados que con estas. En cuanto al HUE, este aumenta el desempeño en los casos de texturas, ya que no tienen cambio de color, por lo que el HUE les entrega esa variación. Mientras que en los fondos de solo color en donde los tonos son una amplia gama, este no representa una mejora.
- **Modificadores:** en la gráfica 4.11 se observa que los modificadores de *blur*, *crop* y *noise* con parámetros adecuados ayudan a mejorar la calidad del entrenamiento. Por otro lado, se tiene que los modificadores que representar mayor influencia en la calidad son el de *noise* y el aleatorizador de color e intensidad, ya que al sacarlos el rendimiento decae 0.101 y 0.067 respectivamente.

Con base en estos experimentos se determina que los mejores parámetros para la generación del *dataset* son los siguientes: distancia 0.6, agrupación de 80 %, aleatorizador de color de iluminación y modificadores de *noise*, *blur* y *crop* con valores 5 %, 5p y 20 % respectivamente.

5.4. Análisis de calidad de los *datasets*

Para los resultados de calidad de los *datasets* generados primero se efectúa un análisis sobre las curvas de precisión versus confianza, precisión versus *recall*, *recall* versus confianza, F1 versus confianza y matriz de confusión para

cada *dataset*. Luego, se realizan observaciones generales del desempeño de las clasificaciones realizadas en todos los *datasets*.

5.4.1. Curvas de precisión versus confianza

Los gráficos de precisión versus confianza indican lo que ocurre con la precisión al aumentar el umbral de confianza en la red. El comportamiento que suele dar es que al aumentar el umbral, también aumenta la precisión, ya que es más probable que los positivos sean verdaderos.

Con base en las gráficas del anexo B.2.1 se tienen las siguientes observaciones:

- **Dataset real:** se observa que entre los puntos de confianza 0.8 y 1.0 las curvas entregan una precisión de 1, esto ocurre, ya que no hay bacterias detectadas en estos rangos. Por otro lado, los bacilos y los cocos tienen un decaimiento en la curva para valores cercanos a los puntos de confianza 0.8 y 0.9. Esto quiere decir que con rango más bajo se detectaban aciertos verdaderos, pero al subir el rango estos se filtraron dejando más falsos positivos. Mientras que para los espirilos, la relación entre precisión y confianza se mantiene más pareja en la curva.
- **Dataset sintético:** se observa que entre los puntos 0.8 y 1.0 de confianza hay un decaimiento para las tres curvas. Esto ocurre con mayor intensidad en los cocos, ya que para un valor de confianza cercano al 0.9 se filtraron los verdaderos positivos, dejando solo falsos aciertos, por lo que la precisión decae a 0.
- **Dataset mixto:** se ve una mejora en los decaimientos abruptos de precisión en todas las curvas con respecto a los casos real y sintético. Además, se observa que para las tres curvas se dejan de detectar bacterias en el valor de confianza de 0.9 aproximadamente.

5.4.2. Curvas de precisión versus *recall*

El gráfico de precisión versus *recall* indica lo que sucede con la detección de verdaderos positivos en relación con la capacidad de detectar positivos sin considerar si son o no verdaderos. Cuanto mayor sea la precisión, más confianza tendrá el modelo cuando clasifique una muestra como positiva. Al

contrario, mientras mayor sea el *recall*, más muestras positivas clasificará el modelo como positivo.

Con base en las gráficas del anexo B.2.2 se tienen las siguientes observaciones:

- **Dataset real:** en la gráfica se observa que la mayor tasa de aciertos con un bajo número de *recall* son los espirilos, por consiguiente, cuando se detectaron pocos espirilos la mayoría fueron aciertos verdaderos. Por la naturaleza de la curva de precisión versus *recall*, se tiene que a mayor número de detecciones menor es la precisión. Este principio se cumple para las tres curvas del gráfico.
- **Dataset sintético:** en las curvas se ve que para cierto valor de *recall* la precisión comienza a decaer con mayor intensidad. En el caso de los cocos esta se produce aproximadamente en el valor de 0.2 de *recall*, es decir, para cantidades bajas detectadas se tiene un mínimo de 30 % de falsos aciertos. La curva que le sigue son los bacilos, en los cuales la precisión empieza a decaer cerca del 0.4 de *recall*. Finalmente, el que tiene mejores valores es el espirilo, el cual tiene su caída de precisión cerca del 0.6 de *recall*.
- **Dataset mixto:** se ve que con un número bajo de *recall* se tienen valores altos de precisión. El que tiene peor desempeño son los bacilos, iniciando su precisión cerca del 0.8. Además, se observa que las tres clases tienen un buen equilibrio entre precisión y *recall*, es decir, a mayor número de detecciones, la precisión disminuye en una proporción similar.

5.4.3. Curvas de *recall* versus confianza

Este gráfico indica qué sucede con la capacidad de detectar positivos con relación al umbral de confianza. Como la confianza penaliza a los falsos positivos, y a mayor valor de *recall* la probabilidad de un falso positivo aumenta, esto quiere decir que a mayor confianza se tendrá menor *recall*.

Con base en las gráficas del anexo B.2.3 se tienen las siguientes observaciones:

- **Dataset real:** en este gráfico se observa que las tres clases tienen una curva de similar comportamiento, en donde el *recall* decae a mayor

confianza. La curva que disminuye con mayor intensidad es la de los cocos, la cual llega a *recall* 0 cerca del punto de confianza con valor 0.8. En general, se ve que desde el valor de confianza 0.5 las curvas decaen con mayor intensidad.

- **Dataset sintético:** la principal diferencia que se observa en las curvas del gráfico es desde donde inicia el *recall*. Se tiene que los cocos inician en 0.6, los bacilos en 0.8 y los espirilos en 0.9. Con esto, se concluye que los cocos tienen una menor cantidad de detecciones en general a comparación de los bacilos y espirilos.
- **Dataset mixto:** en la gráfica se observa que los cocos tienen un aumento en la cantidad de detecciones a comparación de los casos real y sintético. Este supera a los espirilos entre los umbrales de confianza con valores desde 0 hasta 0.2. Por otro lado, los bacilos permanecen por debajo de la curva de los espirilos en todo el umbral de confianza.

5.4.4. Curvas de F1 versus confianza

El gráfico del puntaje F1 en relación con la confianza indica cómo se comporta el equilibrio entre capacidad y precisión en reconocer positivos comparados con el umbral de confianza que se utilice en la detección.

Con base en las gráficas del anexo B.2.4 se tienen las siguientes observaciones:

- **Dataset real:** en este gráfico se tiene que las tres clases inician con valor de F1 cercano al 0.2 y crecen hasta aproximadamente el 0.8. Luego en el umbral de confianza cercano al 0.7 este tiene una caída hasta llegar al valor F1 igual a 0. Es decir, para valores de confianza entre 0 y 0.7 se tiene un buen equilibrio entre el número de detecciones y su precisión. Además, se observa que la clase que mantiene mejores valores de F1 son los bacilos, luego los espirilos y, finalmente, los cocos.
- **Dataset sintético:** se observa que las curvas aumentan hasta un valor F1 máximo y luego descienden hasta llegar a F1 igual a 0 en valores de confianza cercanos a 1.0. Este punto máximo para los bacilos, cocos y espirilos ocurre aproximadamente en los valores 0.5, 0.4 y 0.7 respectivamente. Dado esto, se concluye que los cocos tiene mayores diferencias entre precisión y *recall* para todos los valores del umbral de confianza en comparación con las demás formas.

- **Dataset mixto:** en la curva se ve que en valores bajos de confianza el valor F1 aumenta, luego llega a un punto máximo de equilibrio y decae hasta el valor de F1 igual a 0. Se tiene que este punto es de 0.6 para los bacilos, 0.7 para los cocos y 0.8 para los espirilos. Por otro lado, en comparación al *dataset* real, el punto máximo de los cocos y espirilos aumenta mientras que el de los bacilos disminuye.

5.4.5. Matrices de confusión

La matriz de confusión indica el porcentaje que hay para una confusión entre la clase predicha y su valor real. Esta matriz incluye como clase el *background* que se considera como todo lo que no son objetos principales.

Con base en las gráficas del anexo B.2.5 se tienen las siguientes observaciones:

- **Dataset real:** se observa que los bacilos tienen un 45 % de probabilidad de que sean confundidos con el fondo, a este le siguen los cocos con un 38 % y, finalmente, los espirilos con un 18 %. Esto se justifica debido a que la forma de los espirilos es más representativa y no suele confundirse con el ruido del fondo. Por otro lado, no se consideran confusiones considerables entre las formas, ya que el valor máximo de confusión ocurre en los espirilos y los bacilos con un 2 %.
- **Dataset sintético:** se observa que la confusión de las bacterias con el fondo para los bacilos, cocos y espirilos es de 37 %, 41 % y 22 % respectivamente. Por lo que se mantienen similares al caso real, a excepción de los espirilos que aumentan un 19 %. Por otro lado, no se consideran confusiones determinantes entre las formas bacterianas, ya que su máximo valor ocurre entre los cocos y los bacilos con un 3 %. Por otro lado, existe una cantidad considerable de falsos positivos en los cocos con un valor del 36 % de que un coco predicho sea verdadero.
- **Dataset mixto:** en la gráfica se tiene que los falsos positivos de los cocos disminuye en comparación al caso real y el sintético. Este tiene un valor de 85 % de que un coco predicho realmente tenga esta forma. Mientras que las otras dos formas, bacilos y espirilos, tienen un 62 % y 79 % respectivamente de que la forma predicha sea la verdadera. Por otro lado, la confusión de las bacterias con el fondo es de aproximadamente 46 % para los bacilos, 41 % para los cocos y 13 % para los

espirilos, por lo que se mantiene a comparación del *dataset* real.

5.4.6. Observaciones generales

En la gráfica de la figura 5.1 se entrega un resumen del desempeño de los entrenamientos. En este se incorpora a la gráfica la detección sobre el dominio de los sintéticos para los *datasets* real, sintético y mixto.

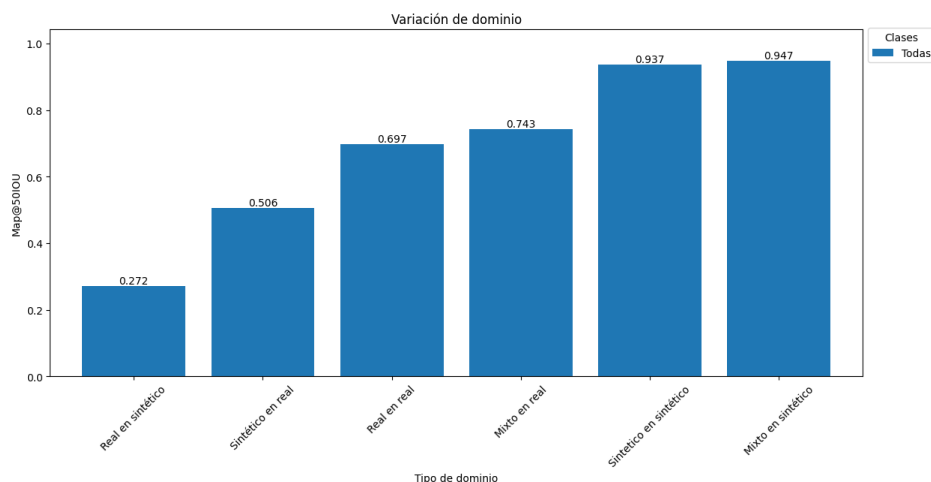


Figura 5.1. Gráfica de variación de dominio que entrega el mAP50 general para todas las formas del modelo. Este señala que el mejor *dataset* es el mixto, seguido del real y, finalmente, el sintético, además se observa que el desempeño de los dominios sintético y mixto detectando bacterias sintéticas es bastante alto.

Si bien el *dataset* real tiene mejor desempeño que el sintético, este es superado por el mixto. Por lo que es posible para emplearlo en *data augmentation*.

Por otro lado, la diferencia entre el *dataset* real y el sintético es de aproximadamente 0.171. Considerando que el *dataset* de *test* es reducido, es posible que con mayores diferencia en el *dataset* de testeo el sintético supere al real, ya que este tiene mayor aleatoriedad en sus propiedades. Como también, añadir nuevos aleatorizadores del dominio de microbiología e incorporar diseños de bacterias puede aumentar la calidad del entrenamiento y disminuir el *domain gap* en los resultados.

En cuanto a la clasificación de las bacterias, de las tablas 4.5, 4.6 y 4.7 se tiene que en general los espirilos tienen mejor clasificación, estos tienen formas alargadas y curvas por lo que difieren de las formas esféricas y cilíndricas. Mientras que el que tiene menor valor de mAP son los cocos, esto se debe a que las formas esféricas son fácilmente confundibles con ruido de las

imágenes, y si estas se dividen, entonces pasan a ser similares a los cilindros, por lo que son más difíciles de clasificar e identificar del entorno.

Por último, en la sección 3.2 se definió un valor de 80 % como mínimo de detección para tener una cantidad de bacterias representativa de la colonia. En este caso es posible determinar la capacidad de detección de los modelos implementados observando el mAP, ya que contempla entre sus variables la cantidad y precisión de las detecciones. Esta tiene su mayor valor en el *dataset* mixto con 0.743, por lo que no alcanza el mínimo estipulado. Las razones de esto se deben al elevado nivel de ruido y cantidad de bacterias por imagen que dificulta la tarea incluso para modelos de detección entrenados con datos reales.

Capítulo 6

Conclusión y trabajos futuros

En esta investigación se planteó la posibilidad de generar un *dataset* sintético a partir de un motor gráfico para realizar identificación y clasificación de bacterias en arquitecturas de tipo YOLO. Esto con la finalidad de entregar una herramienta que permita obtener datos de bacterias independientes de tener acceso a un laboratorio y a los conocimientos básicos de bacteriología.

En esta se utiliza el módulo Perception de Unity en donde se diseñaron e implementaron diversos modelos 3D para el escenario de la generación de datos.

Con base en pruebas realizadas en la estructura de YOLOv5 se mide la calidad de la generación de datos obtenida por el simulador. Para esto se compara su rendimiento con un *dataset* real y uno mixto utilizando un *dataset* de testeo independiente a los utilizados en los entrenamientos.

Lo obtenido en las pruebas responde de forma parcial a la hipótesis planteada, ya que el *dataset* sintético entrega un rendimiento deteriorado en comparación al entrenamiento con datos reales. Mientras que al utilizar la técnica de extensión con datos reales se tiene un *dataset* mixto con calidad de entrenamiento mayor al dominio real. Es decir, es posible emplear los datos generados en *data augmentation*.

En los experimentos realizados se determina que al utilizar un modelo con enfoque a la aleatorización de dominio, el modelo optimiza sus valores a un conjunto general, en donde la realidad es solo un subconjunto. Y por ende, este no necesariamente se enfocará en mejorar hacia ese tipo de datos. Es por

esto que se hace más difícil saber qué parámetros mejoran el entrenamiento hacia el objetivo.

Existen varias formas de lidiar con esto, la primera es incluir más factores de aleatoriedad que se adecúen al dominio, como la inclusión de esporas y flagelos en los modelos. La segunda es utilizar un conjunto de validación real para hacer que el entrenamiento mejore hacia el subconjunto requerido, pero esto afectaría su característica de ser un conjunto en su totalidad libre de datos reales.

Por último, se concluye que la generación de datos mediante una simulación representa una solución que ofrece un potencial de mejora continuo al permitir la inclusión de nuevos modelos, texturas, aleatorizadores y modificadores. Con esto, el *dataset* sintético cada vez se acercará más a los resultados obtenidos con datos reales.

Referencias bibliográficas

- [1] M. Pérez and M. Mota, “Morfología y estructura bacteriana,” *Temas de bacteriología y virología médica*, pp. 23–42, 2006.
- [2] D. J. Jiménez, “Identificación semántica de objetos mediante Deep Learning,” 2018.
- [3] A. Chakure. (2021) Fully connected layer in a CNN. <https://dev.to/afrozchakure/cnn-in-a-brief-27gg>.
- [4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” 2015. [Online]. Available: <https://arxiv.org/abs/1506.02640>
- [5] A. Sharma, “Introduction to the YOLO family,” in *PyImageSearch*, D. Chakraborty, P. Chugh, A. R. Gosthipaty, S. Huot, K. Kidriavsteva, R. Raha, and A. Thanki, Eds., 2022, <https://pyimg.co/dgbvi>.
- [6] A. Voulodimos, N. Doulamis, A. Doulamis, and E. Protopapadakis, “Deep learning for computer vision: A brief review,” *Computational Intelligence and Neuroscience*, vol. 2018, pp. 1–13, 02 2018.
- [7] A. Bochkovskiy, C. Wang, and H. M. Liao, “Yolov4: Optimal speed and accuracy of object detection,” *CoRR*, vol. abs/2004.10934, 2020. [Online]. Available: <https://arxiv.org/abs/2004.10934>
- [8] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,” 2022. [Online]. Available: <https://arxiv.org/abs/2207.02696>
- [9] N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” 2017. [Online]. Available: <https://arxiv.org/abs/1703.07402>
- [10] K. Emam, L. Mosquera, and R. Hoptroff, *Practical Synthetic Data Generation: Balancing Privacy and the Broad Availability of Data*. O’Reilly Media, Incorporated, 2020. [Online]. Available: <https://books.google.cl/books?id=C9ejzAEACAAJ>

REFERENCIAS BIBLIOGRÁFICAS

- [11] V. Danilov, O. Gerget, D. Kolpashchikov, N. Laptev, R. Manakov, L. Hernández-Gómez, F. Alvarez, and M. Ledesma-Carbayo, “Boosting segmentation accuracy of the deep learning models based on the synthetic data generation,” *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. XLIV-2/W1-2021, pp. 33–40, 04 2021.
- [12] Unity Technologies, “Unity Perception package,” <https://github.com/Unity-Technologies/com.unity.perception>, 2020.
- [13] G. Roberto C., E. Briseida, C. Marian, and O. Manuel, *Las tinciones básicas en el Laboratorio de Microbiología: Un enfoque gráfico.*, 2020.
- [14] N. Mohamad, N. Jusoh, Z. Zaw, and S. Win, “Bacteria identification from microscopic morphology: A survey,” *International Journal on Soft Computing, Artificial Intelligence and Applications*, vol. 3, pp. 1–12, 05 2014.
- [15] M. Berlanga, “Microbiología (5a edn). lansing m. prescott, john p. harley, donald a. klein,” *International Microbiology; Vol. 7, Núm. 3 (2004); 234*, 01 2010.
- [16] A. Musto, *Manual De Microbiología Y Parasitología*, Editorial Universidad Nacional Arturo Jauretche, 2013.
- [17] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org>.
- [18] Y. S. Abu-Mostafa, M. Magdon-Ismail, and H.-T. Lin, *Learning From Data*. AMLBook, 2012.
- [19] R. D. Saúl, “Estudio de la arquitectura YOLO para la detección de objetos mediante deep learning,” Master’s thesis, Escuela Técnica Superior de Ingenieros de Telecomunicación. Universidad de Valladolid, 2021.
- [20] J. Nelson. (2021) Your comprehensive guide to the YOLO family of models. <https://blog.roboflow.com/guide-to-yolo-models/>.
- [21] S. Hinterstoisser, O. Pauly, H. Heibel, M. Marek, and M. Bokeloh, “An annotation saved is an annotation earned: Using fully synthetic

- training for object instance detection.” [Online]. Available: <https://arxiv.org/abs/1902.09967>
- [22] S. Hinterstoisser, V. Lepetit, P. Wohlhart, and K. Konolige, “On pre-trained image features and synthetic images for deep learning,” 2017. [Online]. Available: <https://arxiv.org/abs/1710.10710>
- [23] J. Shermeyer, T. Hossler, A. Van Etten, D. Hogan, R. Lewis, and D. Kim, “Rareplanes: Synthetic data takes flight,” 2020. [Online]. Available: <https://arxiv.org/abs/2006.02963>
- [24] K. Bousmalis, N. Silberman, D. Dohan, D. Erhan, and D. Krishnan, “Unsupervised pixel-level domain adaptation with generative adversarial networks,” 2016. [Online]. Available: <https://arxiv.org/abs/1612.05424>
- [25] A. Gupta, A. Vedaldi, and A. Zisserman, “Synthetic data for text localisation in natural images,” 2016. [Online]. Available: <https://arxiv.org/abs/1604.06646>
- [26] S. James, P. Wohlhart, M. Kalakrishnan, D. Kalashnikov, A. Irpan, J. Ibarz, S. Levine, R. Hadsell, and K. Bousmalis, “Sim-to-real via sim-to-sim: Data-efficient robotic grasping via randomized-to-canonical adaptation networks,” 2018. [Online]. Available: <https://arxiv.org/abs/1812.07252>
- [27] L. Duan, D. Xu, and I. Tsang, “Learning with augmented features for heterogeneous domain adaptation,” 2012. [Online]. Available: <https://arxiv.org/abs/1206.4660>
- [28] A. Rozantsev, M. Salzmann, and P. Fua, “Beyond sharing weights for deep domain adaptation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 4, pp. 801–814, apr 2019. [Online]. Available: <https://doi.org/10.1109/TPAMI.2018.2814042>
- [29] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” 2017.
- [30] Marco Clavero Suazo, “Detección de células para su posterior simulación, a partir de material gráfico estático y dinámico,” 2021.

REFERENCIAS BIBLIOGRÁFICAS

- [31] R. Padilla, W. L. Passos, T. L. B. Dias, S. L. Netto, and E. A. B. da Silva, “A comparative analysis of object detection metrics with a companion open-source toolkit,” *Electronics*, vol. 10, no. 3, 2021. [Online]. Available: <https://www.mdpi.com/2079-9292/10/3/279>
- [32] Tzutalin, “Labelimg,” Free Software: MIT License, 2015. [Online]. Available: <https://github.com/tzutalin/labelImg>
- [33] B. Dwyer, K. Nelson, and J. Solawetz, “Roboflow (Version 1.0),” 2022. [Online]. Available: <https://roboflow.com>.
- [34] Alexey, J. Redmon, S. Sinigardi, cyy, T. Hager, JaledMC, M. Maaz, V. Zhang, J. Alasuutari, P. Kahn, IlyaOvodov, J. Veitch-Michaelis, A. Dujardin, J. Aughey, A. Patel, duohappy, Aven, D. Smith, J. White, MarvinKlemp, E. Özipek, A. Babaei, B. Gąsiorzewski, Dev-Nash, Double, HagegeR, G. Daras, I. Popovych, J. A. Pizzorno, and M. Giordano, “Alexeyab/darknet: Yolov4,” Oct. 2021. [Online]. Available: <https://doi.org/10.5281/zenodo.5622675>
- [35] T. Lin, M. Maire, S. J. Belongie, L. D. Bourdev, R. B. Girshick, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: common objects in context,” *CoRR*, vol. abs/1405.0312, 2014. [Online]. Available: <http://arxiv.org/abs/1405.0312>
- [36] G. Jocher, “YOLOv5 by Ultralytics,” 5 2020. [Online]. Available: <https://github.com/ultralytics/yolov5>

ANEXOS

Anexo A

Primer anexo

A.1. Conversión de archivos de anotaciones para YOLOv4

Código A.1. Conversión de archivos de anotaciones para YOLOv4

```
1 # Importaciones necesarias
2 import json
3 import os
4 image_path = '/Darknet/data'
5
6 # Creación de archivo classes.txt
7 with open(image_path + 'Dataset/annotation_definitions.json', 'r') as
    file:
8     classes_json_file = json.load(file)
9
10 with open(image_path + "RGB/classes.txt", 'w') as classes_file:
11     for classe in classes_json_file['annotation_definitions'][0]['spec'
        ]:
12         classes_file.write(
13             '%s\n' %(
14                 classe['label_name']
15             )
16         )
17
18 # Crear anotaciones .txt en formato YOLO para cada imagen
19 pic_height = 640
20 pic_width = 640
21
22 dir_list = os.listdir(image_path + 'Dataset/')
```

```

23 captures_list=[str for str in dir_list if any(sub in str for sub in [
    "captures"])]
24
25 for capture in captures_list:
26     with open(image_path + 'Dataset/' + capture , 'r') as file:
27         big_json_file = json.load(file)
28     for picture in big_json_file['captures']:
29         filename = picture['filename'].split('/')[0]
30         filename = filename[:4] + '.txt'
31         with open(image_path + 'RGB/' + filename, 'w') as
            annotation_file:
32             for bbox in picture['annotations'][0]['values']:
33                 annotation_file.write(
34                     '%d_%f_%f_%f\n' % (
35                         bbox['label_id'],
36                         (bbox['x'] + bbox['width']/2) / pic_width,
37                         (bbox['y'] + bbox['height']/2) / pic_height,
38                         bbox['width'] / pic_width,
39                         bbox['height'] / pic_height
40                     )
41                 )

```

A.2. Creación de archivos de entrenamiento para YOLOv4

Código A.2. Creación de archivos de entrenamiento para YOLOv4

```

1 # Importaciones necesarias
2 import os
3
4 # Dirección de la carpeta generada por Unity
5 image_path = '/Darknet/data/RGB'
6 os.chdir(image_path)
7
8 # Iteración por cada imagen del directorio y guardado de la dirección
   de cada imagen en el arreglo path_list
9 path_list = []
10 for current_dir, dirs, files in os.walk('.'):
11     for f in files:
12         if f.endswith('.jpg'):
13             file_loc = image_path + '/' + f
14             path_list.append(file_loc + '\n')
15
16 # División de los datos en un ratio de 80:20
17 # Se obtienen 20% de los datos y se guardan en el archivo de testeo
18 path_list_test = path_list[:int(len(path_list) * 0.20)]
19
20 # Se eliminan 20% de los datos seleccionados como testeo, quedan 80%
   de datos para entrenamiento

```

```
21 path_list = path_list[int(len(path_list) * 0.20):]
22
23 # Creación de archivo train.txt y escritura del 80% de los datos
24 with open('train.txt', 'w') as train:
25     for i in path_list:
26         train.write(i)
27
28 # Creación de archivo test.txt y escritura del 20% de los datos
29 with open('test.txt', 'w') as test:
30     for i in path_list_test:
31         test.write(i)
32
33 # Inicialización de contador
34 i = 0
35
36 # Crear archivo classes.names desde el contenido existente de classes
    .txt
37 with open(image_path + '/' + 'classes.names', 'w') as cls, \
38     open(image_path + '/' + 'classes.txt', 'r') as text:
39     for l in text:
40         cls.write(l)
41         i += 1
42
43 # Creación de archivo image_data.data
44 with open(image_path + '/' + 'image_data.data', 'w') as data:
45     # Escritura del número de clases
46     data.write('classes_{}_ '.format(i) + '\n')
47
48     # Escritura de la dirección del archivo train.txt
49     data.write('train_{}_ '.format(image_path + '/' + 'train.txt') + '\n')
50
51     # Escritura de la dirección del archivo test.txt
52     data.write('valid_{}_ '.format(image_path + '/' + 'test.txt') + '\n')
53
54     # Escritura de la dirección del archivo classes.names
55     data.write('names_{}_ '.format(image_path + '/' + 'classes.names') + '\n')
56
57     # Especificación de la dirección de la carpeta donde se guardarán
        los pesos entrenados del modelo
58     data.write('backup_{}_ '.format('backup'))
```

A.3. Conversión de anotaciones y estructuración de carpetas para YOLOv5

Código A.3. Creación de anotaciones y estructura para YOLOv5

```
1 # Importaciones necesarias
2 import json
3 import os
```

PRIMER ANEXO

```
4
5 # Dirección hacia la carpeta Dataset generada por Unity
6 dataset_path = '/result/'
7
8 # Dirección hacia la carpeta de destino
9 dataset_path_final = '/result/files/'
10
11 # Creación de la carpeta para las anotaciones
12 import os.path
13 from os import path
14
15 if path.exists( dataset_path_final) == False:
16     os.mkdir(dataset_path_final)
17
18     os.mkdir(dataset_path_final + 'labels')
19     os.mkdir(dataset_path_final + 'labels/train')
20     os.mkdir(dataset_path_final + 'labels/val')
21     os.mkdir(dataset_path_final + 'labels/test')
22
23     os.mkdir(dataset_path_final + 'images')
24     os.mkdir(dataset_path_final + 'images/train')
25     os.mkdir(dataset_path_final + 'images/val')
26     os.mkdir(dataset_path_final + 'images/test')
27
28 # Creación de anotaciones .txt y estructuración en formato YOLOv5
    para cada imagen
29
30 pic_height = 640
31 pic_width = 640
32
33 dir_list = os.listdir(dataset_path + 'Dataset/')
34 captures_list=[str for str in dir_list if any(sub in str for sub in [
    "captures"])]
35
36 total_images = len(os.listdir(dataset_path + 'RGB/'))
37 i = 0
38 ratio_val = 0.2
39
40 for capture in captures_list:
41     with open(dataset_path + 'Dataset/' + capture , 'r') as file:
42         big_json_file = json.load(file)
43     for picture in big_json_file['captures']:
44         filename = picture['filename'].split('/')[ -1]
45         filename = filename[: -4]
46         i = i + 1
47         if( i <= total_images * ratio_val):
48             sub_path = "val/"
49         else:
50             sub_path = "train/"
51         with open(dataset_path_final + 'labels/' + sub_path + filename
            + '.txt', 'w') as annotation_file:
52             for bbox in picture['annotations'][0]['values']:
53                 annotation_file.write(
```

```

54         '%d_%f_%f_%f_%f\n' % (
55             bbox['label_id'],
56             (bbox['x'] + bbox['width']/2) / pic_width,
57             (bbox['y'] + bbox['height']/2) / pic_height,
58             bbox['width'] / pic_width,
59             bbox['height'] / pic_height
60         )
61     )
62     os.rename(dataset_path + 'RGB/' + filename + '.png',
63               dataset_path_final + 'images/' + sub_path + filename + '.png'
64             ')
65
66 # Creación de archivo .yaml
67 # Acceso a las classes
68 with open(dataset_path + 'Dataset/annotation_definitions.json', 'r')
69     as file:
70     classes_json_file = json.load(file)
71
72 i = 0
73 classes = ''
74 for classe in classes_json_file['annotation_definitions'][0]['spec']:
75     classes = classes + "'" + classe['label_name'] + "',"
76     i = i + 1
77
78 dir_name = 'train_data'
79
80 data_yaml = """
81 # Train/val/test sets as 1) dir: path/to/imgs, 2) file: path/to/imgs.
82   txt, or 3) list: [path/to/imgs1, path/to/imgs2, ..]\n
83 # path: ../"" + dir_name + "" # dataset root dir\n
84 train: ../"" + dir_name + ""/images/train/ # train images (
85   relative to 'path') 128 images\n
86 val: ../"" + dir_name + ""/images/val/ # val images (relative to '
87   path') 128 images\n
88 test: # test images (optional)\n
89 \n
90 # Classes\n
91 nc: "" + str(i) + "" # number of classes\n
92 names: ["" + classes[:-1] + ""] # class names\n"""
93
94 with open(dataset_path_final + 'data.yaml', 'w') as annotation_file:
95     annotation_file.write(data_yaml)

```

A.4. Aleatorizadores personalizados

A.4.1. Aleatorizador de posición de objetos principales orientada a la agrupación

Código A.4. Aleatorizador de posición de objetos principales orientada a la agrupación

```

1 using System;
2 using System.Collections.Generic;
3 using System.Diagnostics;
4 using System.Linq;
5 using UnityEngine.Perception.Randomization.Parameters;
6 using UnityEngine.Perception.Randomization.Randomizers.Utilities;
7 using UnityEngine.Perception.Randomization.Samplers;
8
9 namespace UnityEngine.Perception.Randomization.Randomizers.
    SampleRandomizers
10 {
11     [Serializable]
12     [AddRandomizerMenu("Bacterias/ForegroundArrangementPlacementRandomizer")]
13     public class ForegroundArrangementPlacementRandomizer :
        Randomizer
14     {
15         [Tooltip("The Z offset applied to positions of all placed objects.")]
16         public float depth;
17
18         [Tooltip("The minimum distance between the centers of the placed objects.")]
19         public float separationDistance = 2f;
20
21         [Tooltip("The width and height of the area in which objects will be placed. These should be positive numbers and sufficiently large in relation with the Separation Distance specified.")]
22         public Vector2 placementArea;
23
24         [Tooltip("The list of Prefabs to be placed by this Randomizer.")]
25         public GameObjectParameter prefabs;
26
27         [Tooltip("The probability for the division of bacteria.")]
28         public float ArrangementProb = 50;
29
30         GameObject m_Container;
31         GameObjectOneWayCache m_GameObjectOneWayCache;
32
33         protected override void OnAwake()
34         {
35             m_Container = new GameObject("ForegroundObjects");
36             m_Container.transform.parent = scenario.transform;
37             m_GameObjectOneWayCache = new GameObjectOneWayCache(
38                 m_Container.transform, prefabs.categories.Select(
39                     element => element.Item1).ToArray(), this);
40         }
41
42         protected override void OnIterationStart()
43         {

```



```

43     var seed = SamplerState.NextRandomState();
44     var placementSamples = PoissonDiskSampling.
        GenerateSamples(
45         placementArea.x, placementArea.y, separationDistance,
            seed);
46     var offset = new Vector3(placementArea.x, placementArea.y
        , 0f) * -0.5f;
47     bool division = false;
48     var divisionCounter = 0f;
49     var prevPlacementSample = new Vector3(placementSamples
        [0].x, placementSamples[0].y, depth);
50     var prevAngleDivision = -1f;
51     bool makeDivision = false;
52     var probOfDivision = new UniformSampler(0, 100);
53     var angleSample = new UniformSampler(0, 180);
54     var angle = 1000f;
55     float w = 0f;
56     var newPost = new Vector3(0, 0, 0);
57     const float PI = (float)Math.PI;
58     List<Vector3> listSamples = new List<Vector3>();
59     List<float> listAngles = new List<float>();
60
61     foreach (var sample in placementSamples)
62     {
63         var instance = m_GameObjectOneWayCache.
            GetOrInstantiate(prefabs.Sample());
64         if (division && (probOfDivision.Sample() <
            ArrangementProb - (20 * divisionCounter)))
65         {
66             makeDivision = true;
67             divisionCounter++;
68         }
69         else if (listSamples.Count > 0 && probOfDivision.
            Sample() < 30)
70         {
71             makeDivision = true;
72             prevPlacementSample = listSamples[0];
73             listSamples.RemoveAt(0);
74             prevAngleDivision = listAngles[0];
75             listAngles.RemoveAt(0);
76             divisionCounter = 1;
77         }
78         if (makeDivision)
79         {
80             angle = angleSample.Sample();
81             listAngles.Add(angle);
82             if ((angle < prevAngleDivision - 70 && angle >
                prevAngleDivision + 70))
83             {
84                 angle = -angle;
85             }
86             var width = instance.GetComponentInChildren<
                Renderer>()[0].bounds.size.x;

```

```

87         prevAngleDivision = Math.Abs(angle);
88         var angleRad = PI * (angle / 180f);
89         newPost = prevPlacementSample + new Vector3(width
            * Mathf.Cos(angleRad), width * Mathf.Sin(
            angleRad), depth);
90         instance.transform.position = newPost + offset;
91         listSamples.Add(newPost);
92         makeDivision = false;
93     }
94     else
95     {
96         prevAngleDivision = 1000f;
97         instance.transform.position = new Vector3(sample.
            x, sample.y, depth) + offset;
98         prevPlacementSample = new Vector3(sample.x,
            sample.y, depth);
99         division = true;
100        divisionCounter = 0f;
101    }
102 }
103 listSamples.Clear();
104 listAngles.Clear();
105 placementSamples.Dispose();
106 }
107 protected override void OnIterationEnd()
108 {
109     m_GameObjectOneWayCache.ResetAllObjects();
110 }
111 }
112 }

```

A.4.2. Aleatorizador de color e intensidad de la iluminación

Código A.5. Aleatorizador de color e intensidad de la iluminación

```

1 using System;
2 using UnityEngine.Perception.Randomization.Parameters;
3 using UnityEngine.Perception.Randomization.Randomizers.
    SampleRandomizers.Tags;
4 using UnityEngine.Perception.Randomization.Samplers;
5
6 namespace UnityEngine.Perception.Randomization.Randomizers.
    SampleRandomizers
7 {
8     [Serializable]
9     [AddRandomizerMenu("Bacterias/Light_Color_Randomizer")]
10
11     public class LightColorRandomizer : Randomizer
12     {
13         [Tooltip("The light of the scene.")]
14         public Light light;

```

```

15
16     [Tooltip("The range of intensity value.")]
17     public FloatParameter intensity = new FloatParameter { value
        = new UniformSampler(0, 1) };
18
19     [Tooltip("The range of R value.")]
20     public FloatParameter r = new FloatParameter { value = new
        UniformSampler(0, 1) };
21
22     [Tooltip("The range of G value.")]
23     public FloatParameter g = new FloatParameter { value = new
        UniformSampler(0, 1) };
24
25     [Tooltip("The range of B value.")]
26     public FloatParameter b = new FloatParameter { value = new
        UniformSampler(0, 1) };
27
28     [Tooltip("The range of A value.")]
29     public FloatParameter a = new FloatParameter { value = new
        UniformSampler(0, 1) };
30
31     protected override void OnIterationStart()
32     {
33         Color newColor = Color.white;
34         newColor.r = r.Sample();
35         newColor.g = g.Sample();
36         newColor.b = b.Sample();
37         newColor.a = a.Sample();
38         light.color = newColor;
39         light.intensity = intensity.Sample();
40     }
41 }
42 }

```

Anexo B

Segundo anexo

B.1. Muestra de imágenes del *dataset* sintético.



Figura B.1. Selección de escenas aleatorias utilizadas en la fase de entrenamiento para la comparación de *datasets*.

B.2. Gráficas de comparación entre *dataset* real, sintético y mixto.

B.2.1. Gráficas de precisión versus confianza

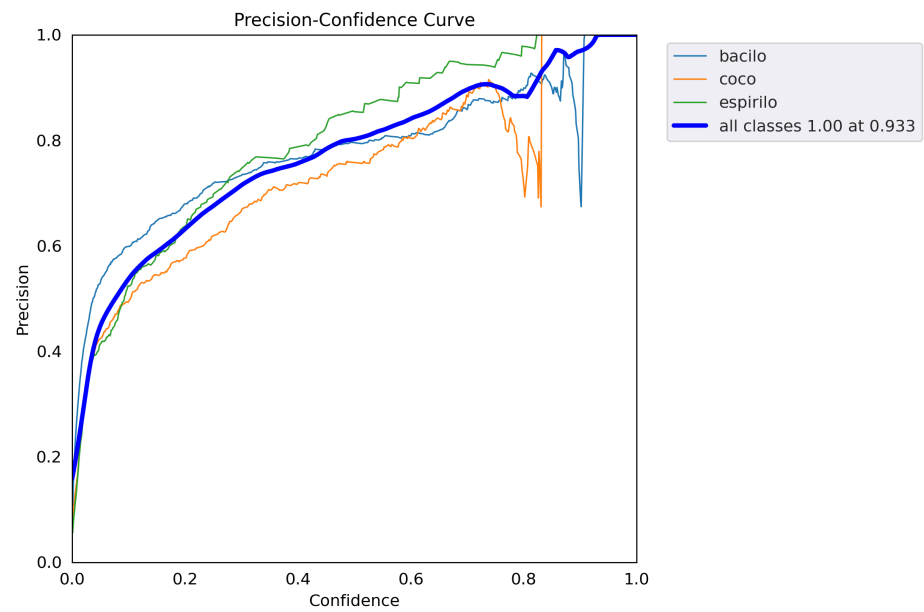


Figura B.2. Curva de precisión versus confianza para el *dataset* real.

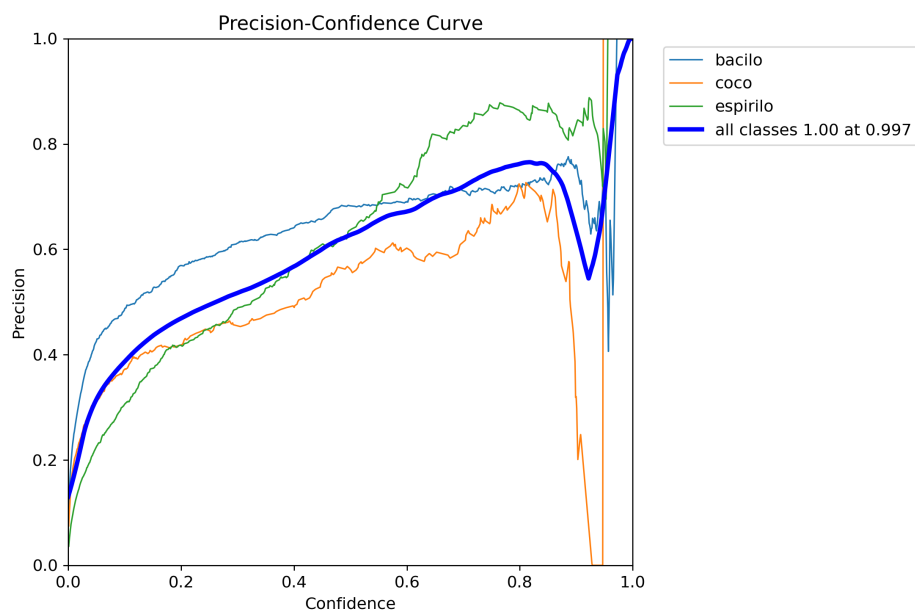


Figura B.3. Curva de precisión versus confianza para el *dataset* sintético.

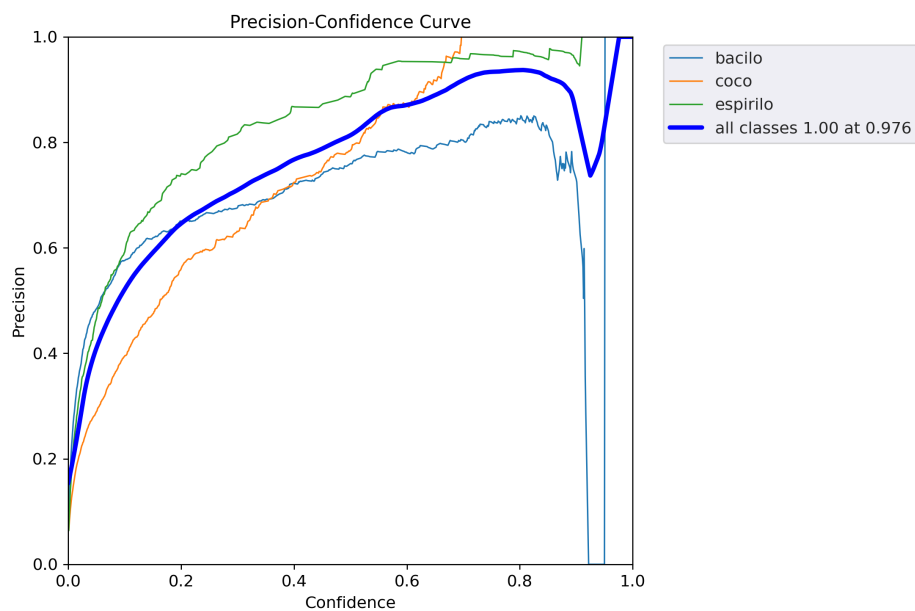


Figura B.4. Curva de precisión versus confianza para el *dataset* mixto.

B.2.2. Gráficas de precisión versus *recall*

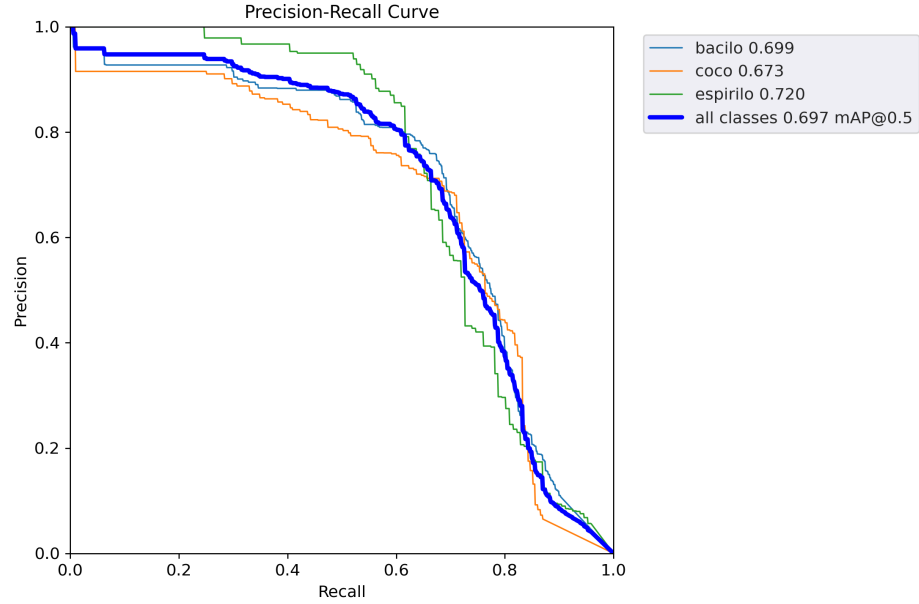


Figura B.5. Curva de precisión versus *recall* para el *dataset* real.

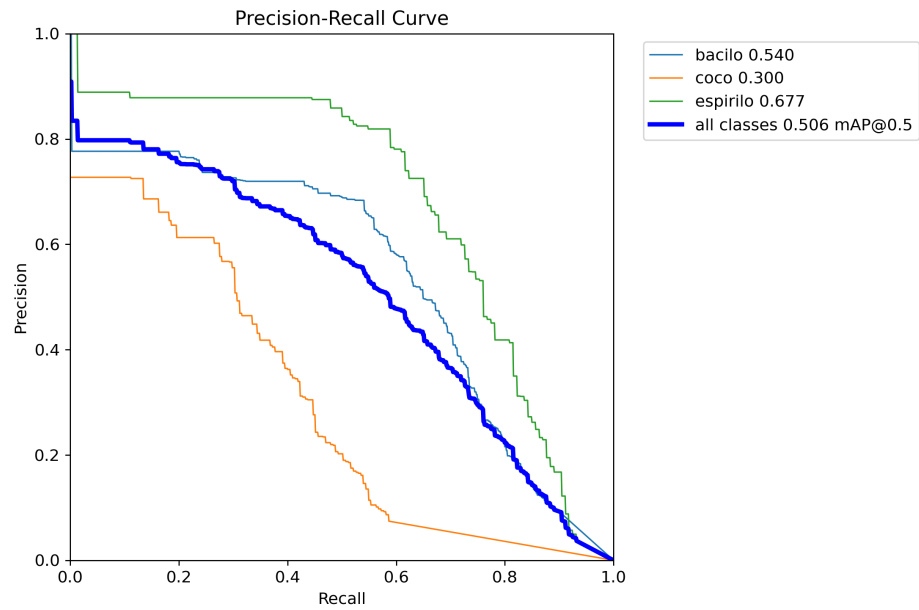


Figura B.6. Curva de precisión versus *recall* para el *dataset* sintético.

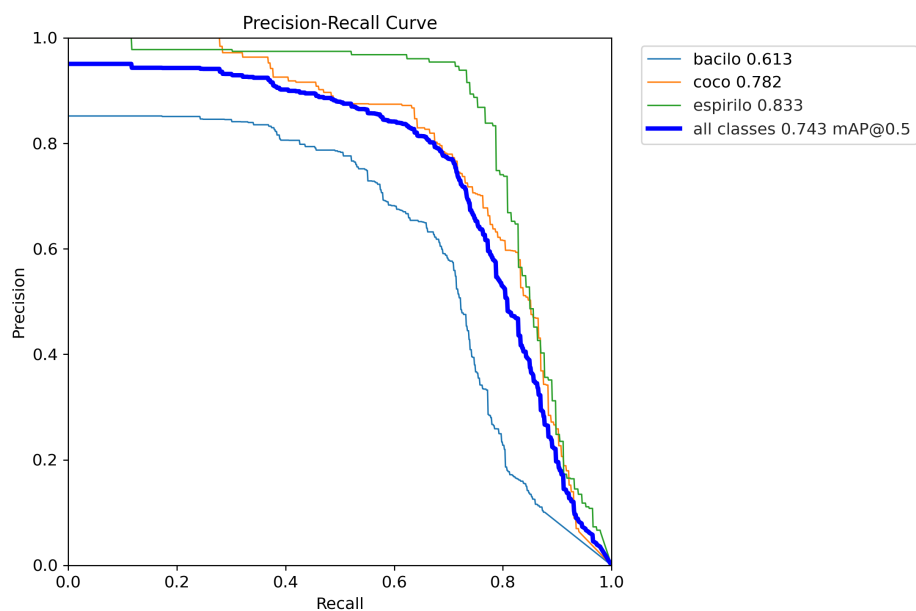


Figura B.7. Curva de precisión versus *recall* para el *dataset* mixto.

B.2.3. Gráficas de *recall* versus confianza

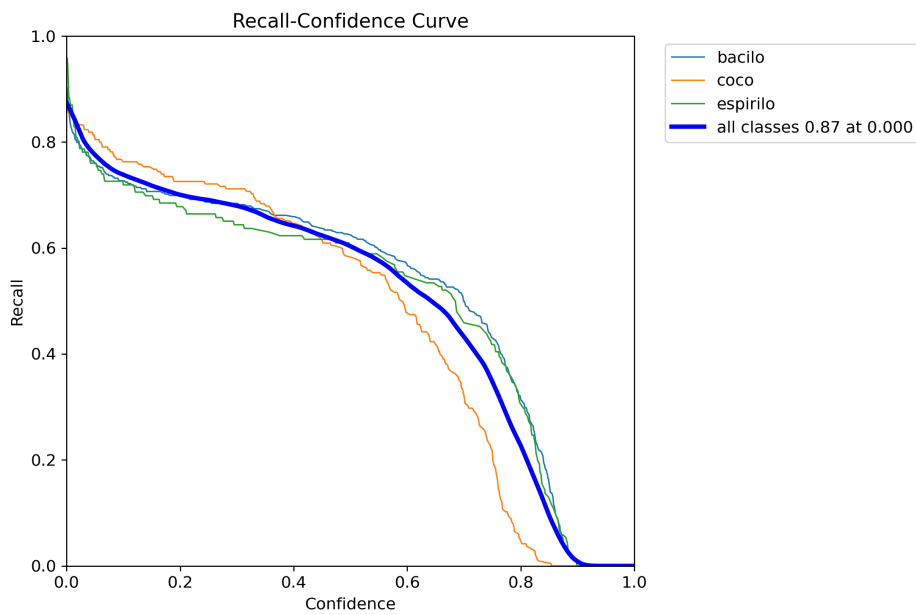


Figura B.8. Curva de *recall* versus confianza para el *dataset* real.

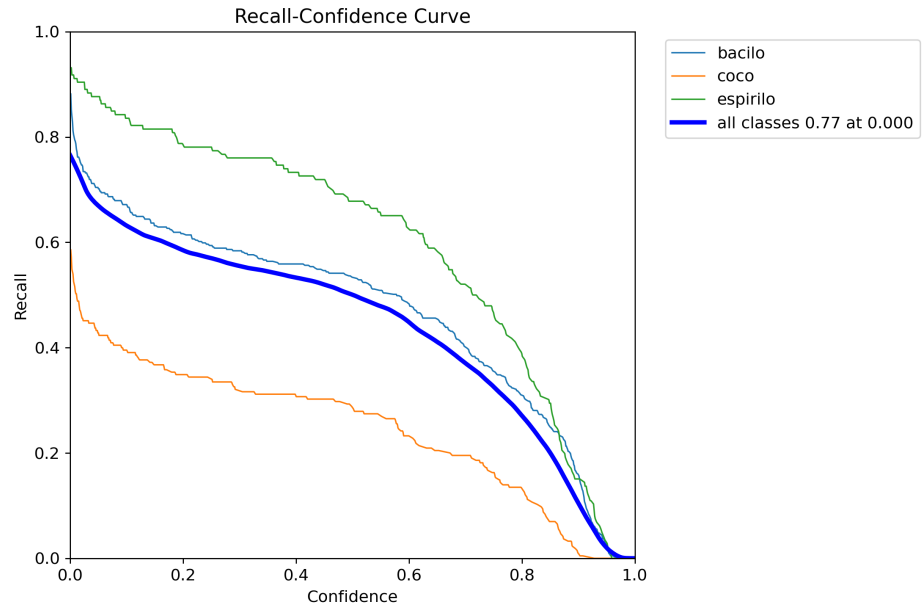


Figura B.9. Curva de *recall* versus confianza para el *dataset* sintético.

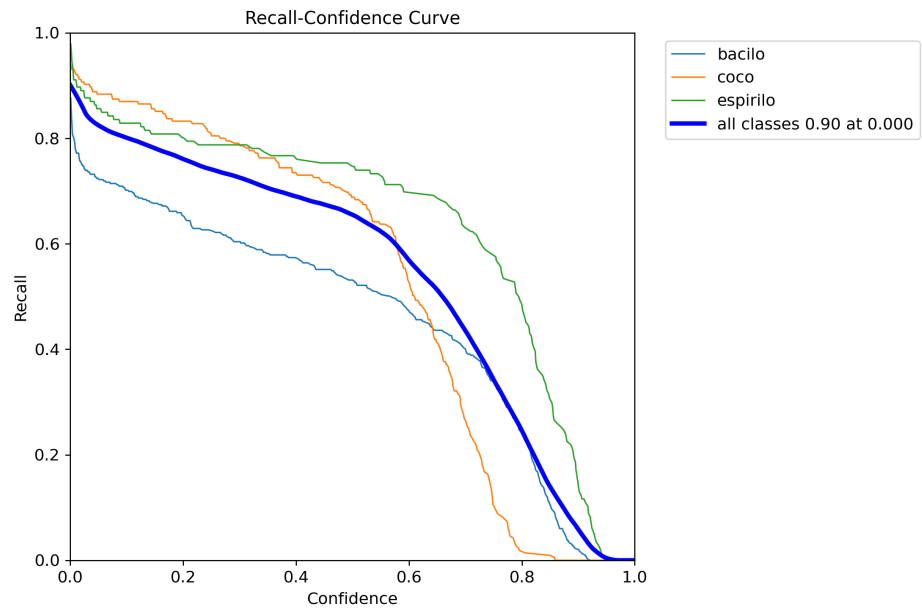


Figura B.10. Curva de *recall* versus confianza para el *dataset* mixto.

B.2.4. Gráficas de F1 versus confianza

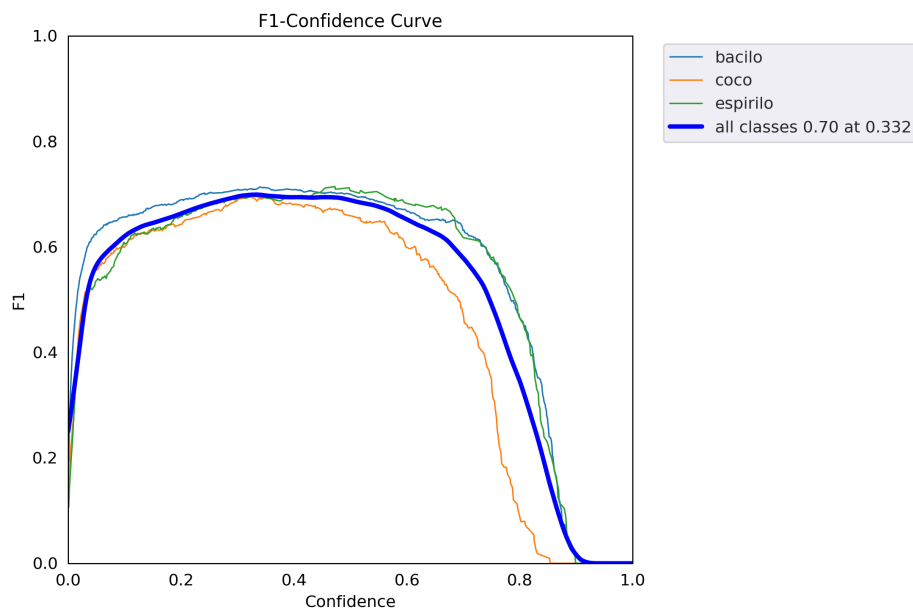


Figura B.11. Curva de F1 versus confianza para el *dataset* real.

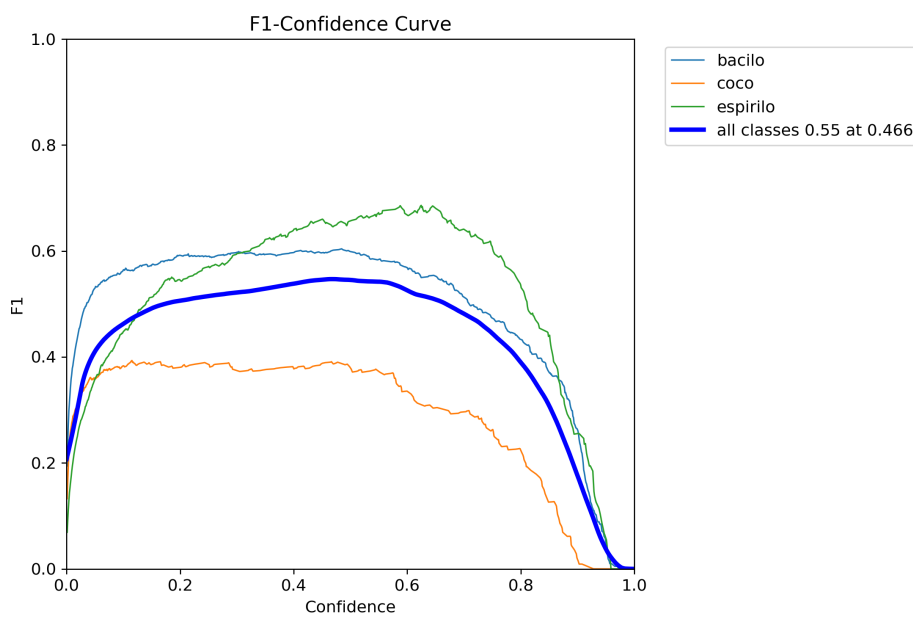


Figura B.12. Curva de F1 versus confianza para el *dataset* sintético.

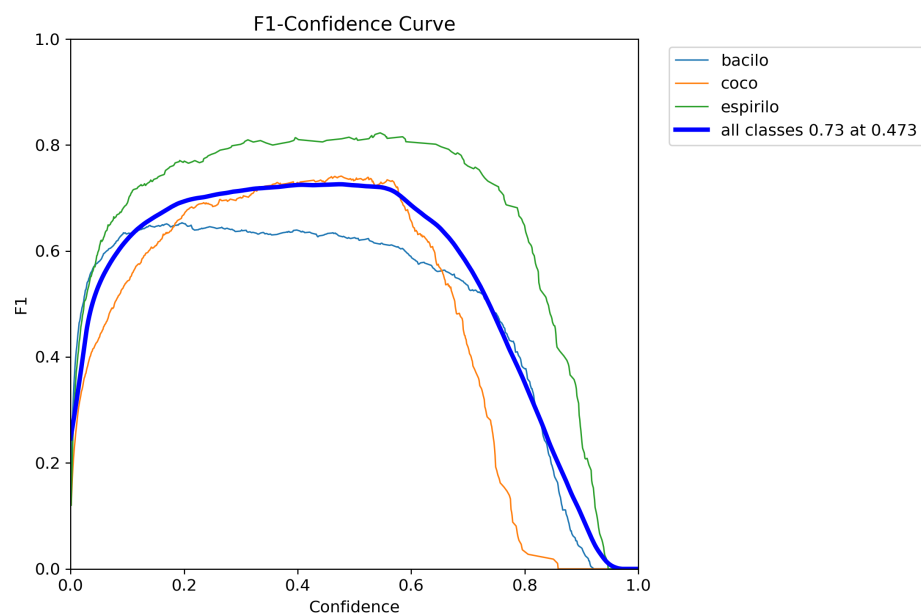


Figura B.13. Curva de F1 versus confianza para el *dataset* mixto.

B.2.5. Matrices de confusión

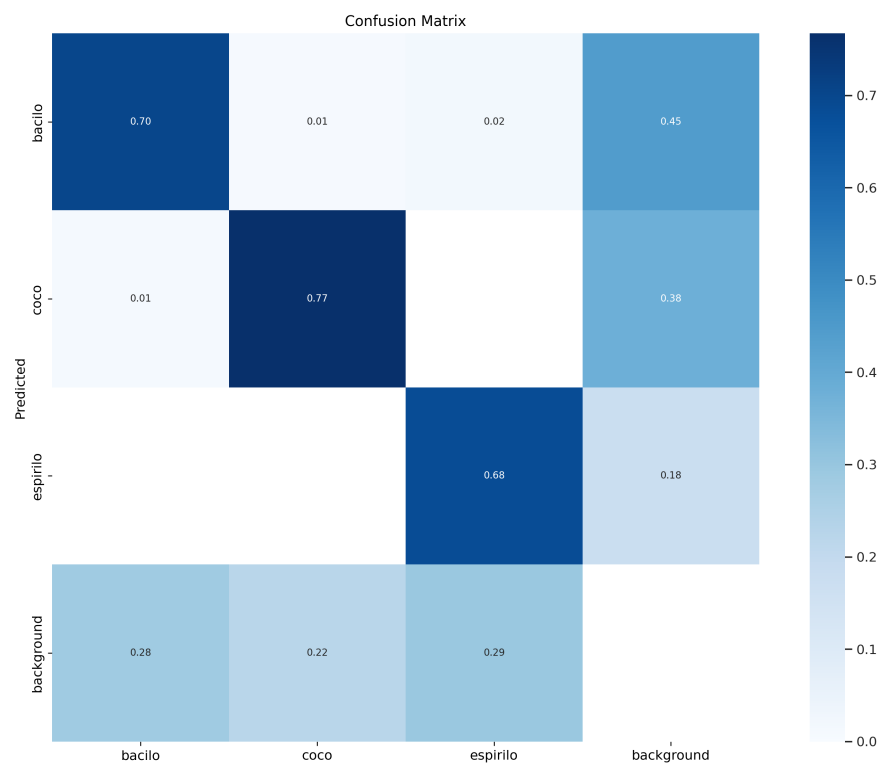


Figura B.14. Matriz de confusión para el *dataset* real.

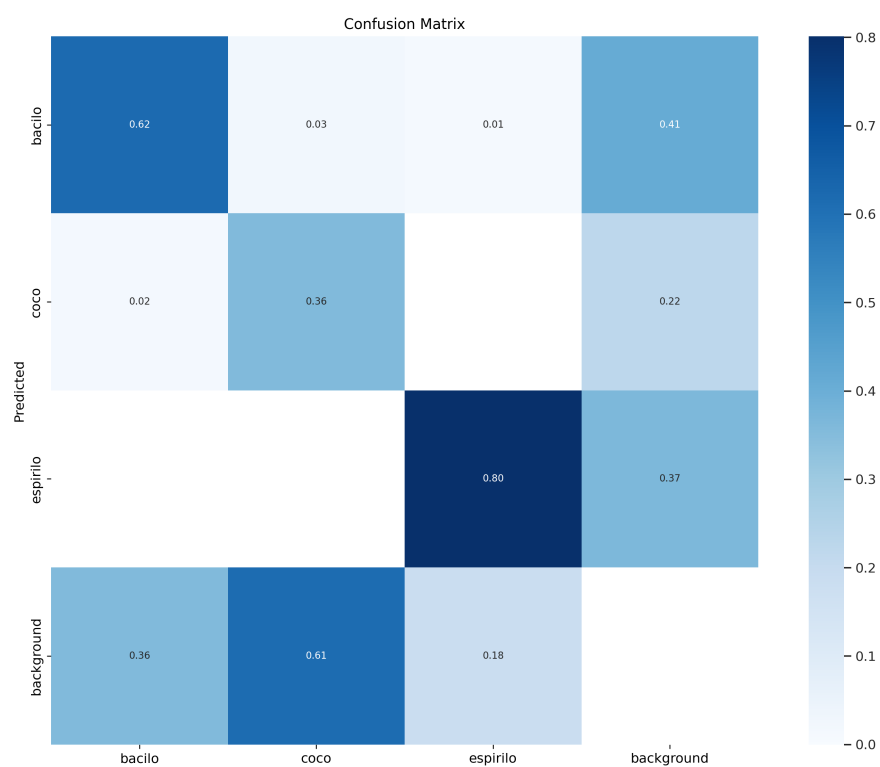


Figura B.15. Matriz de confusión para el *dataset* sintético.

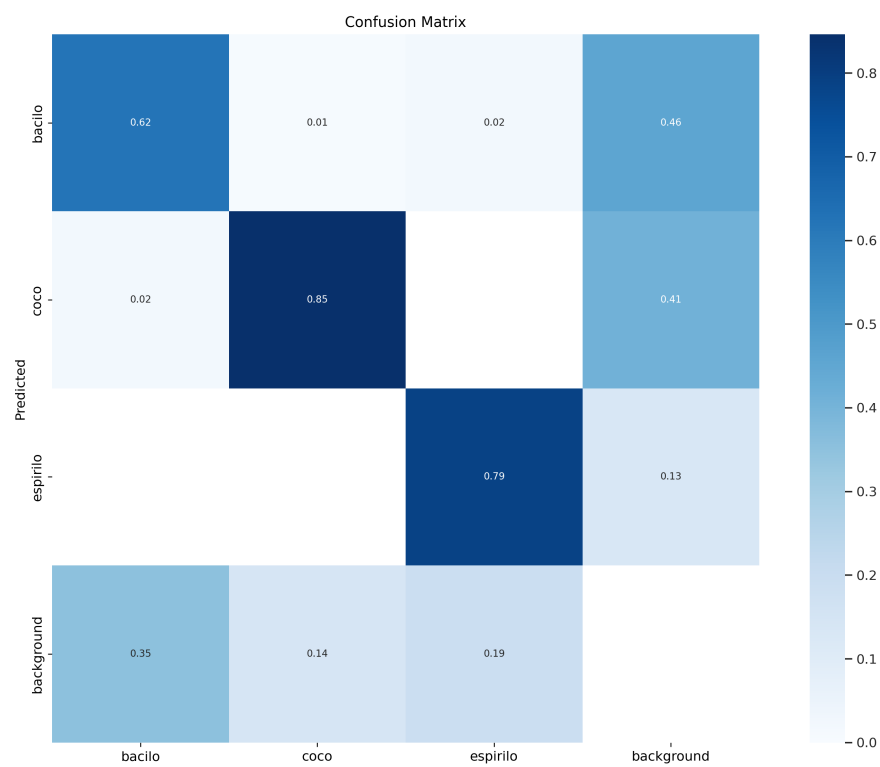


Figura B.16. Matriz de confusión para el *dataset* mixto.